

MOPRO SHOP

Tek VDS'den Ölçeklenebilir Mimariye - Tam Planlama Dökümanı

Hibrit Modular Monolith + Continuous Ledger Sürümü

Hazırlayan: Mimari ve Ürün Planlama Ekibi

Sürüm 3.2 · Mayıs 2026 · Hardening + Stratejik Eki Dahil

Hedef Kitle: Kurucu, CTO, Mobil ve Backend Geliştiriciler, Operasyon Ekibi

İçindekiler

İçindekiler	2
1. Yönetici Özeti	9
1.1 Stratejik Tasarım Prensipleri	9
1.2 Maliyet Anchor'ı	9
1.3 Doküman Haritası	9
2. İş Modeli ve Stratejik Konumlandırma	11
2.1 Pazar Tanımı	11
2.2 Değer Önerileri	11
<i>Satıcı Tarafı</i>	11
<i>Tüketici Tarafı</i>	11
2.3 Gelir Modeli ve Float Yönetimi	11
2.4 Rakip Konumlandırması	12
3. Ürün Vizyonu, Hedefler ve OKR'lar	13
3.1 Vizyon Cümlesi	13
3.2 12 Aylık Stratejik Hedefler	13
3.3 OKR Çerçevesi (İlk Çeyrek)	13
<i>Objective 1 — Sağlam Finansal Çekirdek</i>	13
<i>Objective 2 — Kullanıcı ve Satıcı Büyümesi</i>	13
<i>Objective 3 — Operasyonel Olgunluk</i>	13
4. MVP Kapsamı ve Kapsam Dışı	14
4.1 Kapsam İçi (MVP'de Olacak)	14
4.2 Kapsam Dışı (V2'ye Bırakıldı)	14
5. Mobil Uygulama Mimarisi (Flutter)	15
5.1 Katmanlı Mimari	15
5.2 State Management ve Veri	15
5.3 Flutter Paket Önerileri	15
5.4 Tasarım Sistemi	15

5.5 Dağıtım ve Sürüm Yönetimi	16
6. Ekran Akışları (User Flows)	17
6.1 Onboarding ve Kimlik	17
6.2 Keşif	17
6.3 Ürün Detay ve Beden Bulucu	17
6.4 Sepet ve Ödeme	17
6.5 Cüzdan	17
6.6 Müşteri Hizmetleri	17
7. Sistem Mimarisi - Yüksek Seviye Bakış	18
7.1 Bootstrap-First Felsefesi	18
7.2 Mantıksal Katmanlar	18
7.3 Mimari Diyagram (Metinsel)	18
7.4 Trafiğin Tipik Akışı	18
8. Teknoloji Yığını	20
8.1 Bootstrap Yığını (MVP)	20
8.2 Scale-Up Yığını (İleride)	20
9. Mikroservis Sınırları ve Sorumlulukları	22
9.1 Servisler Arası İletişim Kuralları	22
10. Veri Modeli	23
10.1 E-Ticaret Tarafı (Özet Şema)	23
10.2 Cüzdan ve Ledger Tarafı (Append-Only, AYRI DB)	23
10.3 Hesap Planı (Chart of Accounts)	24
11. Olay Akışları ve Saga Pattern	25
11.1 Sipariş -> Komisyon -> Coin Akışı	25
11.2 EventBus Interface'i (Go)	26
11.3 Telafi Stratejileri	26
11.4 Olay Sözleşmeleri	27

12. API Tasarım Prensipleri	28
12.1 Örnek Endpoint Sözleşmesi	28
13. Tek VDS Donanım Profili ve Kaynak Bütçesi	29
13.1 Donanım	29
13.2 RAM Bütçesi (24 GB)	29
13.3 Disk Bütçesi (120 GB)	30
13.4 CPU Bütçesi (6 vCPU)	30
13.5 Network Bütçesi	30
14. Docker Compose ile Tam Kurulum	31
14.1 Dizin Yapısı	31
14.2 docker-compose.yml (Çekirdek)	32
15. Caddy Reverse Proxy ve TLS	37
15.1 Caddyfile	38
15.2 Caddy'nin Kazandırdıkları	39
16. PostgreSQL Dual Kurulum (Ecom + Ledger İzolasyonu)	41
16.1 İzolasyon Disiplinleri	41
16.2 postgres-ecom postgresql.conf (Optimize)	42
16.3 postgres-ledger postgresql.conf (Daha Sıkı)	43
16.4 Init Scripts	43
16.5 Migration Disiplini	44
17. Redis, Meilisearch ve PgBouncer Konfigürasyonu	45
17.1 Redis	45
17.2 Meilisearch	45
17.3 PgBouncer	45
17.4 Connection Topolojisi	46
18. Backup Stratejisi	47
18.1 Hedefler	47

18.2 Backup Mimarisi	47
18.3 Backup Script (Bash)	48
18.4 WAL Archiving (Saatlik)	49
18.5 Restore Tatbikatı (HAFTALIK Zorunlu)	49
19. Gözlemlenebilirlik	51
19.1 Stratejik Tercih	51
19.2 Grafana Agent (Tek Toplayıcı)	52
19.3 SLO Tanımları	53
19.4 Alarm Hiyerarşisi	53
20. CloudFlare CDN, WAF ve DDoS Koruması	54
20.1 Yapılandırma Adımları	54
20.2 Cache Kuralları	54
20.3 WAF Kuralları	54
20.4 DDoS Koruması	54
21. Dağıtık Mimariye Hazırlık (Distributed-Ready Disiplinler)	55
21.1 12 Disiplin	55
21.2 İdempotency-Key Örnek Implementation	56
21.3 Outbox Pattern (Finansal Olaylar İçin)	57
22. İkinci Sunucuya Geçiş Planı	58
22.1 Tetikleyiciler	58
22.2 Hedef Topoloji (2 Sunucu)	58
22.3 Adım Adım Geçiş (Önerilen Bakım Penceresi: Pazar 03:00-07:00)	58
22.4 Üç Sunucu Topolojisi (Reklam Öncesi)	59
22.5 Geçiş Risklerini Azaltma	59
23. Scale-Up: Kubernetes ve Production-Grade Süreçler	60
23.1 Production-Grade Container İmajları	60
23.2 Production Dockerfile (Go Servis)	61

23.3 Kubernetes Deployment Manifesti (Sample)	62
23.4 GitOps ile Deploy (ArgoCD)	63
23.5 Scale-Up Tetikleyicileri	63
24. Güvenlik Mimarisi	64
24.1 Kimlik ve Yetkilendirme	64
24.2 Veri Güvenliği	64
24.3 KVKK Uyumu	64
24.4 Uygulama Güvenliği	64
24.5 Sırların Yönetimi	64
25. CI/CD ve Deploy	65
25.1 Pipeline Akışı	65
25.2 GitHub Actions Workflow Örneği	66
25.3 Rollback Stratejisi	67
26. Maliyet Analizi	68
26.1 Sabit Aylık Maliyet (MVP)	68
26.2 İşlem Bazlı Maliyetler	68
26.3 İkinci Sunucuya Geçiş Sonrası	68
26.4 12 Aylık Senaryo	69
27. Geliştirme Yol Haritası	70
27.1 Önerilen Ekip	70
28. KPI'lar ve Başarı Kriterleri	71
28.1 Teknik KPI'lar	71
28.2 Ürün KPI'ları	71
28.3 Finansal KPI'lar	71
29. Risk Analizi ve Azaltma Stratejileri	72
29.1 Birincil Riskler	72
29.2 İkincil Riskler	72

29.3 Lansman Öncesi Zorunlu Doğrulamalar	72
--	----

30. Operasyonel Süreçler 73

30.1 On-Call ve Incident	73
30.2 Runbook Örneği - Çekim Takılı	73
30.3 Bakım Penceresi Disiplini	73

31. Production Hardening Eki (Sürüm 3.1) 74

31.1 Container Resource Limit'leri ve OOM Koruması	74
<i>Güncellenmiş Compose Şablonu</i>	75
<i>RAM Bütçesinin Yeniden Hesabı</i>	76
31.2 Kernel ve Container Hardening	77
<i>Sürekli Kernel Patching (unattended-upgrades)</i>	77
<i>Container Hardening Kontrol Listesi</i>	78
<i>VDS Saldırı Yüzeyini Daraltma</i>	78
31.3 Disk Yönetim Otomasyonu	79
<i>Haftalık Cleanup Cron</i>	79
<i>Disk Alarm Eşikleri ve Panic Mod</i>	79
<i>WAL Fallback (B2 Erişilemediğinde)</i>	80
31.4 Olay Bus Görünürlüğü ve Replay Aracı	81
1. <i>Tracing İlk Günden Zorunlu</i>	81
2. <i>mopro CLI - Operatör Aracı</i>	81
3. <i>Saga Görünürlük Dashboard'u</i>	82
31.5 Sonsöz: Pragmatizm ve Disiplin Dengesi	83

33. Mimari ve Bütünlük İncelemesi Eki (Sürüm 3.2) 84

33.1 Mimari Yeniden Değerlendirme: Hibrit Modular Monolith	84
<i>Yeni Önerilen Topoloji</i>	84
<i>Eski (12 servis) vs Yeni (3 binary) Karşılaştırma</i>	85
<i>Bu Yaklaşımın Avantajları</i>	85
<i>Modül Sınırları Disiplini</i>	85
33.2 Continuous Ledger Verification	86
<i>Üç Katmanlı Doğrulama</i>	86
<i>Transaction-level Trigger</i>	87
<i>Hourly Reconciliation Job</i>	88
<i>Property-Based Testing (Geliştirme)</i>	88
33.3 Sonsöz	88

34. Sonuç ve Tavsiye

90

1. Yönetici Özeti

Mopro Shop, Türkiye e-ticaret pazarına satıcı odaklı bir değer önerisiyle giren, mobil öncelikli bir pazar yeri uygulamasıdır. Tüketici tarafında Trendyol'un tanınık navigasyonunu referans alırken, satıcı tarafında rakiplerinden tamamen ayrışır: satıcılardan tahsil edilen komisyonlar, TL'ye endeksli kapalı devre bir varlık olan **Mopro Coin** üzerinden satıcılara periyodik olarak iade edilir. Satıcılar bu coinleri platform içi hizmetlerde harcayabilir veya istedikleri zaman nakde çevirebilir.

Şirketin ana gelir modeli; havuzda biriken komisyon ve geçici nakit pozisyonlarının (float) düşük riskli enstrümanlarda değerlendirilmesinden elde edilen getirilerdir. Bu yaklaşım, satıcıyı maliyet kalemi olarak görmek yerine sadakat ortağı haline getirir.

Bu doküman ürün stratejisi, mimari kararlar, **tek VDS üzerinde MVP lansmanı**, dağıtık mimariye geçiş planı, mobil deneyim, güvenlik ve operasyon konularını uçtan uca kapsar. Üç ayağı vardır: (1) finansal bütünlüğü ödün vermeyecek bir cüzdan/ledger çekirdeği, (2) bootstrap aşamasında tek sunucuda çalışan ama dağıtıma hazır bir mimari, (3) iadeleri azaltacak Beden Bulucu ve güven veren hibrit müşteri desteği gibi deneyim diferansiyatörleri.

1.1 Stratejik Tasarım Prensipleri

- **Bootstrap-first, scale-ready:** İlk gün tek VDS'te çalışan ama 1 günde ikinci sunucuya taşınabilen yapı.
- E-ticaret ve cüzdan/ledger verisi ilk günden ayrı Postgres container'larında, ayrı port, ayrı password, ayrı volume.
- Para hareketleri append-only, double-entry ledger üzerinden; satır güncellenmez, ters kayıtla düzeltilir.
- Tüm bağlantılar env değişkeni; localhost yerine container DNS isimleri (12-Factor disiplini).
- Mobil deneyim Trendyol benzeri tanınık IA üzerine inşa edilir; Cüzdan ve Beden Bulucu Mopro'nun stratejik diferansiyatörleridir.
- Reklam dönemi öncesinde ikinci sunucuya geçiş tamamlanır; tek VDS reklamı kaldırmaz.

1.2 Maliyet Anchor'ı

MVP lansmanı için sabit aylık altyapı maliyeti **VDS 940 TL + ek servisler ~300 TL = ~1.250 TL**. PSP komisyonları ve SMS gibi işlem-bazlı kalemler bunun üzerine eklenir ve büyüme ile doğru orantılı artar. Bu tablo Bölüm 23'te detaylandırılmıştır.

1.3 Doküman Haritası

Doküman dokuz ana parçadan oluşur: **Bölüm 2-4** iş modeli ve ürün stratejisi, **Bölüm 5-7** mobil uygulama ve ekran akışları, **Bölüm 8-12** sistem mimarisi ve veri modeli, **Bölüm 13-20** tek VDS üzerinde production kurulum, **Bölüm 21-22** dağıtık mimariye geçiş planı, **Bölüm 23** ileride aktive olacak Kubernetes ve production-grade konteyner süreçleri, **Bölüm 24-30** güvenlik, CI/CD, maliyet, yol haritası, KPI, risk ve operasyon, **Bölüm 31** bağımsız teknik incelemeler ışığında eklenen Production Hardening önlemleri.

v3.1 + v3.2

Sürüm 3.1 değişiklikleri (Bölüm 31): Container resource limit'leri ve OOM koruması, kernel ve container hardening, disk yönetim otomasyonu ve panic mod, olay bus görünürlüğü ve replay aracı. **Sürüm 3.2 değişiklikleri (Bölüm 33):** Mikroservis yerine 3 binary hibrit modular monolith mimari önerisi ve continuous ledger verification (transaction-level trigger + saatlik reconcile). v3.2'de servis sayısı 12'den hibrit 3'e düşürülmüş, FinTech izolasyonu korunmuştur. Hukuki süreçler ve go-to-market konuları bu dökümanın kapsamı dışındadır; ilgili ekipler tarafından yürütülür.

2. İş Modeli ve Stratejik Konumlandırma

2.1 Pazar Tanımı

Türkiye e-ticaret pazarı son beş yılda iki haneli büyüme oranlarını sürdürmektedir. Pazarı hakim oyuncular (Trendyol, Hepsiburada, Amazon TR) genel marketplace modelinde rekabet ederken, satıcı tarafında en sık dile getirilen şikâyet komisyon yükü ve nakit akışı baskısıdır.

Mopro Shop'un birinci hedef kitlesi **küçük ve orta ölçekli satıcılardır**; özellikle giyim, ev tekstili, kişisel bakım ve küçük ev aletleri kategorilerindeki satıcılar başlangıç odaklıdır. Tüketici tarafında hedef kitle mobilde rahat alışveriş yapan, ürün karşılaştırmaya değer veren, özellikle giyim kategorisinde beden uyumu konusunda kaygı taşıyan kullanıcılarıdır.

2.2 Değer Önerileri

Satıcı Tarafı

- Komisyonlar Mopro Coin olarak iade edildiği için satıcı görünür komisyon gideri yerine "erteleme + geri dönüş" yaşar.
- Coin platform içi reklam, öne çıkarma, hızlı ödeme gibi hizmetlerde kullanılabilir; operasyonel maliyetleri düşürür.
- TL'ye endeksli kapalı devre yapı sayesinde satıcı kur veya değer kaybı riskine maruz kalmaz.
- Şeffaf ledger görüntüsü (transaction ID ile izlenebilirlik) finansal güven inşa eder.

Tüketici Tarafı

- Trendyol benzeri tanıdık, mobil-öncelikli, hızlı bir alışveriş deneyimi.
- Beden Bulucu ile giyim alışverişinde iadeyi azaltan akıllı öneri.
- AI destekli + canlı destek hibrit müşteri hizmetleri ile 7/24 hızlı çözüm.
- Satıcıyı destekleyen iş modelinin değer hissi, iletişim kampanyaları için güçlü bir mesaj.

2.3 Gelir Modeli ve Float Yönetimi

Mopro Shop'un birincil gelir kaynağı klasik komisyon değil, **float üzerinden elde edilen finansal getiridir**. Her satışta tahsil edilen komisyon, satıcıya iade edilene veya satıcı tarafından nakde çevrilene kadar şirketin emanet (escrow) banka hesabında veya düşük riskli para piyasası fonlarında değerlendirilir. Türkiye gibi yüksek faiz ortamlarında bu model klasik komisyon modeline kıyasla daha sağlıklı bir nakit akışı yaratabilir.

İkincil gelir kaynakları: (i) satıcılara sunulan reklam ve öne çıkarma hizmetleri, (ii) lojistik komisyonları, (iii) opsiyonel premium üyelik. MVP'de bu kanallar zorunlu değildir; veri modeli ileride desteklenebilir şekilde tasarlanır.

ÖNEMLİ

Float'tan elde edilen getiri muhasebe ve regülasyon açısından dikkatli sınıflandırılmalıdır. Tutarlar lisanslı saklayıcı (custodian) veya banka hesaplarında tutulur, getiri şirket geliri olarak ayrı hesaba yansıtılır. FinTech avukatı görüşü olmadan canlıya geçilmemelidir.

2.4 Rakip Konumlandırması

Boyut	Trendyol/Hepsiburada	Mopro Shop
Komisyon yapısı	Doğrudan kesinti	Tahsil edilir, kapalı devre coin olarak iade
Satıcı bağlılığı	Düşük (kanal değiştirme kolay)	Yüksek (coin ekosistemi içsel kullanım)
Tüketici deneyimi	Olgun, geniş yelpaze	Tanındık IA + Beden Bulucu + Hibrit destek
Gelir modeli	Komisyon + reklam + lojistik	Float getirisi + reklam + lojistik
Regülasyon profili	Klasik e-ticaret + ödeme	E-ticaret + sadakat + emanet hesap denetimi

3. Ürün Vizyonu, Hedefler ve OKR'lar

3.1 Vizyon Cümlesi

"Türkiye'nin satıcılarına nakit akışlarını koruyarak büyüme alanı veren, tüketicilerine ise tanıdık deneyimi akıllı bileşenlerle güçlendiren mobil pazar yeri olmak."

3.2 12 Aylık Stratejik Hedefler

- İlk 12 ayda 500-1.000 aktif satıcı ve 50.000-100.000 aylık aktif tüketici.
- Satıcı 90 günlük tutma oranı %80'in üstünde.
- Beden Bulucu kullanan kullanıcılarda iade oranında en az %15 düşüş.
- Müşteri desteğinde self-servis çözüm oranı %40'ın üstünde.
- Float pozisyonunda tutulan ortalama bakiyenin TL faizinden net pozitif marj.

3.3 OKR Çerçevesi (İlk Çeyrek)

Objective 1 — Sağlam Finansal Çekirdek

- KR1: Ledger reconciliation farkı 12 hafta boyunca = 0.
- KR2: Çekim taleplerinin %95'i 24 saat içinde tamamlanır.
- KR3: Bağımsız mali müşavir tarafından çeyreklik denetim raporu temiz.

Objective 2 — Kullanıcı ve Satıcı Büyümesi

- KR1: 100+ aktif satıcı.
- KR2: 10.000+ aylık aktif kullanıcı.
- KR3: Satıcı NPS > 40, tüketici NPS > 30.

Objective 3 — Operasyonel Olgunluk

- KR1: P95 anasayfa < 800 ms, P95 ödeme adımı < 1.5 sn.
- KR2: Ledger reconciliation job günlük çalışıyor; alarm tek seferlik bile çalmadı.
- KR3: Backup restore tatbikatı haftalık başarılı.

4. MVP Kapsamı ve Kapsam Dışı

4.1 Kapsam İçi (MVP'de Olacak)

- Mobil uygulama (Flutter): iOS + Android
- Tüketici akışları: kayıt, anasayfa, kategori, arama, ürün detay, sepet, ödeme, sipariş takibi
- Beden Bulucu (kural tabanlı v1)
- Hibrit müşteri hizmetleri (makale + ticket + canlı destek köprüsü)
- Cüzdan: bakiye görüntüleme, hareket geçmişi, çekim talebi (alıcı için pasif görünür)
- Satıcı web paneli (basit): mağaza yönetimi, ürün ekleme, sipariş yönetimi, cüzdan, çekim
- Komisyon hesaplama ve aylık settlement
- PSP entegrasyonu (iyzico veya Craftgate)
- OTP ile telefon doğrulama, satıcı için temel KYC

4.2 Kapsam Dışı (V2'ye Bırakıldı)

- Reels tarzı kısa video keşif
- Canlı yayın alışveriş
- Çoklu dil (sadece Türkçe; İngilizce altyapısı hazır)
- Marketplace dışı satıcı tools (CRM, otomatik fiyat ayarlama)
- Tüketici sadakat coini (sadece satıcı tarafı aktif)
- Yurtdışı kargo ve uluslararası satıcı
- Marketplace API (3. parti entegrasyonu)

5. Mobil Uygulama Mimarisi (Flutter)

5.1 Katmanlı Mimari

- **presentation:** ekranlar, widget'lar, Riverpod controller'ları.
- **application:** use case'ler, validation, orkestrasyon.
- **domain:** entity, value object, repository arayüzleri.
- **infrastructure:** dio HTTP client, secure storage, push, analytics.
- Modüller feature bazlı: auth, home, product, cart, wallet, support, settings.

5.2 State Management ve Veri

- Riverpod 2 (riverpod_generator) ile derleme zamanı güvenli provider'lar.
- Repository pattern; offline cache için Hive/Isar; ağ kesintisinde grace period.
- Idempotency-key uygulama tarafında üretilir (UUID v4) ve 24 saat saklanır.
- Push notification (FCM/APNs); kritik bildirimler için fallback SMS.
- Analytics: Mixpanel veya Firebase Analytics; PII gönderilmez, user_id pseudonim ID'dir.

5.3 Flutter Paket Önerileri

Amaç	Paket	Neden
State management	flutter_riverpod + riverpod_generator	Compile-time güvenlik
HTTP	dio + retrofit	Interceptor ekosistemi, kod üretimi
Auth storage	flutter_secure_storage	Keychain/Keystore üzerinden token saklama
Analytics	mixpanel_flutter veya firebase_analytics	Davranış izleme
Crash reporting	sentry_flutter	Stacktrace + breadcrumb
Push	firebase_messaging	FCM + APNs unified
Image	cached_network_image	Disk + memory cache
Local DB	isar veya drift	Offline queue, sepet snapshot
Form	reactive_forms veya formz	Validation

5.4 Tasarım Sistemi

- Tek **MoproTheme**: renk, tipografi, spacing, radius, gölge tokenları.
- Atomic design: token -> atom (Button, Input) -> molecule (ProductCard, WalletCard) -> organism (Cart, Checkout).
- Dark mode hazırlığı baştan; MVP'de kapalı, sürüm 2'de açılır.
- Erişilebilirlik: 4.5:1 kontrast, 44x44 dokunma alanı, ekran okuyucu metinleri.

5.5 Dağıtım ve Sürüm Yönetimi

- iOS: TestFlight -> App Store (manuel onay). Apple Developer: \$99/yıl.
- Android: Internal Track -> Closed -> Open -> Production. Google Play: \$25 tek seferlik.
- Codepush yok; native binary değişiklikleri her zaman store sürümünden gider.
- Force-update gate: minimum desteklenen API versiyonunun altındaki istemci için "güncelleme" zorunluluğu.

6. Ekran Akışları (User Flows)

Mobil deneyimde alt menü sırası: **Anasayfa · Kategoriler · Sepet · Cüzdan · Profil**. Cüzdan, profilin solunda kalıcı ve aktif coin bakiyesini bir badge ile gösterir.

6.1 Onboarding ve Kimlik

- Splash -> Anasayfa misafir modunda gezilebilir.
- Sepete ekleme veya cüzdan dokunması ilk kez auth gate tetikler: telefon + OTP (alıcı) veya tam KYC (satıcı).
- Opsiyonel "Beden Profilim" bottom-sheet ilk açılışta tek seferlik gösterilir.

6.2 Keşif

- Anasayfa: arama bar, banner carousel, kategori chip'leri, kişiselleştirilmiş sıralar, flash indirim.
- Kategori: 2 kolon ürün gridi, sticky filtre (Beden, Marka, Fit, Fiyat).
- Arama: instant suggestions; giyim aramalarında "Senin bedenine uyanları göster" toggle.

6.3 Ürün Detay ve Beden Bulucu

- Galeri, fiyat, varyant seçici, satıcı kartı.
- Beden seçici üstünde **"Bedenimi Bul"** butonu -> bottom-sheet: boy, kilo, fit tercihi, opsiyonel önceki beden.
- Sonuç ekranı: "Senin için M önerilir (güven %78). Daha bol seversen L." — varyant otomatik seçilir.
- Soru-cevap, filtreli yorumlar ("benimle aynı bedendekiler"), satıcı diğer ürünleri.

6.4 Sepet ve Ödeme

- Satıcı bazlı gruplama, kupon, kargo özeti.
- Adres -> ödeme yöntemi (kart, BKM, Apple/Google Pay) -> 3DS -> sipariş onayı.
- Onay ekranında "Bu siparişin komisyonu satıcına Mopro Coin olarak iade edilecek" mikro-rozeti.

6.5 Cüzdan

- Üst kart: Toplam Coin (TL eşitliği görünür) + Para Çek + Detay CTA'ları.
- Sekmeler: Hareketler · Bekleyen Tahakkuklar · Çekimler · Kazanç Raporu.
- Hareket detayında ledger transaction id ve sipariş referansı.
- Çekim: tutar -> banka hesabı -> step-up auth -> bekleyen statü -> tamamlanma push'u.

6.6 Müşteri Hizmetleri

- Yardım Hub'ı: arama, SSS kategorileri, AI Asistan girişi.
- AI niyet anlama -> ilgili makale + aksiyon butonları.
- Çözülemezse tek dokunuşla canlı destek/ticket; bağlam korunur.
- SLA göstergesi: "Ortalama yanıt süresi 4 dk".

7. Sistem Mimarisi - Yüksek Seviye Bakış

7.1 Bootstrap-First Felsefesi

Mopro Shop mimarisi iki paradoks aynı anda çözer: **(a) tek VDS'te çalışacak kadar basit, (b) çoklu sunucuya dağıtılabilecek kadar disiplinli.** İlk gün her şey 6c/24GB/120GB tek bir VDS'te Docker Compose ile çalışır; ama yarın ledger DB'sini ayırmak istediğinde kod değişikliği gerekmez, sadece env değişkeni güncellenir.

7.2 Mantıksal Katmanlar

- **Edge:** CloudFlare CDN + WAF + DDoS koruması (ücretsiz tier).
- **Reverse Proxy:** Caddy; TLS sonlandırma, route, rate limit.
- **Servisler:** Go mikroservisleri, container DNS isimleri ile birbirine bağlı.
- **Veri:** İki ayrı Postgres container (ecom + ledger), Redis, Meilisearch.
- **Mesajlaşma:** Redis Streams (başlangıç) -> Kafka (büyüme).
- **Gözlemlenebilirlik:** Grafana Cloud free tier (logs/metrics dış).
- **Backup:** Backblaze B2'ye günlük encrypted backup.

7.3 Mimari Diyagram (Metinsel)

```
[Mobil Flutter] -- HTTPS --> [CloudFlare CDN+WAF] --> [VDS public IP]
|
[Caddy 443/80]
|
+----- Docker Network -----+
| |
[identity] [catalog] [cart] [order] [...]
| |
+-- Redis Streams (event bus) ----+
| |
[postgres-ecom : 5432] [redis : 6379]
[meilisearch : 7700] [pgbouncer]
|
+-- AYRI: ledger izole --+
|
[postgres-ledger : 5433]
[wallet-svc, treasury-svc]

[grafana-agent] -> Grafana Cloud
[restic backup] -> Backblaze B2
```

7.4 Trafiğin Tipik Akışı

1. Mobil istemci CloudFlare üzerinden VDS'e gelir; statik içerikler CDN'den dönüş yapar.
2. Caddy TLS'i sonlandırır, ilgili Go servisine HTTP route eder.
3. Go servisi kendi DB'sine yazar/okur; gerektiğinde başka servise gRPC/HTTP ile veri ister.

4. Para hareketi gerektiren işlemler bir olayı Redis Streams'e yayınlar; finansal servisler tüketir ve ledger'a kayıt atar.
5. Tüm trafik Grafana Agent ile tek noktadan toplanıp Grafana Cloud'a gönderilir.

8. Teknoloji Yığını

8.1 Bootstrap Yığını (MVP)

Katman	Teknoloji	Gerekçe
Mobil	Flutter 3.x + Riverpod	Tek kod tabanı iOS/Android
Backend	Go 1.22+ (chi/gin)	Hafif binary, container'a uygun
Reverse Proxy	Caddy 2	Otomatik Let's Encrypt, basit config
E-ticaret DB	PostgreSQL 16 (container)	ACID, ilişkisel
Ledger DB	PostgreSQL 16 (AYRI container)	Mantıksal izolasyon
Connection Pool	PgBouncer	Postgres bağlantı tüketimini düşürür
Cache	Redis 7	Sepet snapshot, oturum, idempotency
Event Bus	Redis Streams	MVP için yeterli; Kafka sonra
Arama	Meilisearch	Türkçe, hafif, tek binary
Object Storage	Backblaze B2 (S3 uyumlu)	Görseller, backup
CDN/WAF	CloudFlare Free	Ücretsiz, agresif
Observability	Grafana Cloud Free + Grafana Agent	Logs/metrics dış servis
Hata izleme	Sentry Free	5K event/ay yeterli
Container	Docker + Docker Compose	Tek node yeterli
CI	GitHub Actions	Build + push image

8.2 Scale-Up Yığını (İleride)

Trafiğin gerektireceği zamanda eklenecek bileşenler. Bu liste Bölüm 22'de detaylandırılmıştır.

Katman	MVP	Scale-Up
Orchestration	Docker Compose	Kubernetes (Hetzner Cloud K8s veya AWS EKS)
Event Bus	Redis Streams	Apache Kafka
Arama	Meilisearch	OpenSearch cluster
DB	Tek instance	Multi-AZ + read replica
Service Mesh	Yok	Istio (mTLS)

Katman	MVP	Scale-Up
GitOps	Yok (manuel deploy)	ArgoCD
Image Registry	GitHub Container Registry	ECR + cosign signing
Secret Yönetimi	Docker secrets / .env	External Secrets Operator + Vault

9. Mikroservis Sınırları ve Sorumlulukları

MVP'de 10-12 mikroservis tek VDS'te container'lar olarak çalışır. Her servis kendi DB tablolarına sahiptir; başka servisin tablosunu doğrudan okumaz. İletişim tek node'da bile **container DNS isimleri** üzerindendir; localhost referansı yoktur.

Servis	Sorumluluk	Veri
identity-svc	OTP, oturum, KYC, RBAC	users, sessions, kyc_status
catalog-svc	Ürün, kategori, varyant, stok	products, variants, categories
cart-svc	Sepet, kupon ön-değerlendirme	Redis (kalıcı için Postgres)
order-svc	Sipariş yaşam döngüsü, saga	orders, order_items, shipments
payment-svc	PSP entegrasyonu, 3DS	payments, payment_attempts
seller-svc	Satıcı, mağaza, kontrat	sellers, stores, contracts
commission-svc	Komisyon, settlement	commission_rules, accruals
wallet-svc (FinTech)	Mopro Coin cüzdanı, ledger	wallets, ledger_entries (AYRI DB)
treasury-svc	Float, banka mutabakatı	float_positions, bank_movements
size-finder-svc	Beden öneri	brand_size_charts, fit_models
support-svc	Ticket, AI yönlendirme, makale	tickets, articles
notification-svc	Push, e-posta, SMS	notifications, templates
search-svc	Arama (Meilisearch wrapper)	Meilisearch index
media-svc	Görsel upload, resize	Backblaze B2 + metadata

9.1 Servisler Arası İletişim Kuralları

- Senkron çağrılar: HTTP/JSON tek node'da; ileride gRPC + Istio mTLS.
- Asenkron olaylar: Redis Streams; her olay versiyonlanır (örn. v1.order.completed).
- Veri tutarlılığı: Her servis kendi DB'sine yazar; başka servisten veri istediğinde HTTP çağrısı.
- Hata politikası: Retry sadece idempotent endpoint'lerde; finansal endpoint'lerde idempotency-key zorunlu.
- Kod tarafında EventBus ve Repository interface'leri soyutlanır; implementation Redis Streams -> Kafka geçişinde değişir, kullanıcı kodu değişmez.

10. Veri Modeli

10.1 E-Ticaret Tarafı (Özet Şema)

```
-- Ürün katalog
products(id, seller_id, title, description, brand, category_id, status, created_at)
variants(id, product_id, sku, color, size, price_minor, stock, image_keys)
categories(id, parent_id, name, attributes_schema)

-- Sipariş
orders(id, user_id, status, total_minor, currency, created_at, ...)
order_items(id, order_id, variant_id, seller_id, qty, unit_price_minor,
commission_rule_id)
shipments(id, order_id, courier, tracking_no, status)

-- Ödeme
payments(id, order_id, psp, psp_ref, amount_minor, status, created_at)
payment_attempts(id, payment_id, attempt_no, response_code, latency_ms)
```

10.2 Cüzdan ve Ledger Tarafı (Append-Only, AYRI DB)

Mopro Coin için klasik bir double-entry ledger kullanılır. Bakiye **asla** bir balance kolonundan okunmaz; her zaman ledger_entries üzerindeki D/C farklarından materialized view ile hesaplanır.

```
-- AYRI Postgres container'ında (postgres-ledger:5433)
-- Hesap planı
accounts(
id BIGSERIAL PRIMARY KEY,
type TEXT NOT NULL, -- asset/liability/equity
owner_type TEXT, -- platform/seller/bank
owner_id BIGINT,
currency TEXT DEFAULT 'TRY_COIN',
status TEXT DEFAULT 'active',
created_at TIMESTAMPTZ DEFAULT now()
);

-- Mantıksal işlem
transactions(
id BIGSERIAL PRIMARY KEY,
type TEXT NOT NULL, -- commission_refund, withdraw, reversal, ...
reference TEXT, -- order_id veya başka bir referans
idempotency_key TEXT UNIQUE NOT NULL,
status TEXT DEFAULT 'posted',
created_at TIMESTAMPTZ DEFAULT now()
);
```

```
-- Çift kayıt ledger satırı (APPEND-ONLY)
ledger_entries(
  id BIGSERIAL PRIMARY KEY,
  transaction_id BIGINT NOT NULL REFERENCES transactions(id),
  account_id BIGINT NOT NULL REFERENCES accounts(id),
  direction CHAR(1) NOT NULL CHECK (direction IN ('D','C')),
  amount_minor BIGINT NOT NULL CHECK (amount_minor > 0),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Append-only güvencesi
CREATE RULE no_update_ledger AS ON UPDATE TO ledger_entries DO INSTEAD NOTHING;
CREATE RULE no_delete_ledger AS ON DELETE TO ledger_entries DO INSTEAD NOTHING;

-- Bakiye görüntüsü (materialized view)
CREATE MATERIALIZED VIEW wallet_balances AS
SELECT account_id,
SUM(CASE WHEN direction='C' THEN amount_minor ELSE -amount_minor END) AS
balance_minor
FROM ledger_entries
GROUP BY account_id;

-- Reconciliation invariant kontrol query
SELECT SUM(CASE WHEN direction='C' THEN amount_minor ELSE -amount_minor END) AS total
FROM ledger_entries;

-- Bu sorgu HER ZAMAN 0 dönmelidir.
```

10.3 Hesap Planı (Chart of Accounts)

- **asset:bank:escrow** — Şirketin emanet banka hesabındaki gerçek TL.
- **liability:platform_pool** — Komisyon havuzu (satıcılara borç).
- **liability:wallet:seller_<id>** — Her satıcının cüzdanı.
- **liability:bank_outbound** — Çekim için ayrılmış ama henüz ödenmemiş tutarlar.
- **equity:retained_float_income** — Float'tan elde edilen şirket geliri (treasury reklasifikasyonu sonrası).

PERFORMANS

Cüzdan API'leri her okumada balance'ı materialized view'dan üretir. Yüksek frekansta çağrılan endpoint'ler için Redis'te 5-10 sn cache kullanılır; ancak çekim talebi gibi kritik akışlarda her zaman canlı sorgu yapılır.

11. Olay Akışları ve Saga Pattern

Sipariş oluşturma, ödeme, komisyon hesaplama ve coin iadesi gibi çok adımlı süreçler **distributed saga** ile yönetilir. Saga'nın orchestratörü order-svc'dir; her adım başarısız olduğunda telafi edici işlem tetiklenir.

MVP'de event bus olarak **Redis Streams** kullanılır. Servis kodunda EventBus interface'i soyutlanmıştır; ileride Kafka'ya geçildiğinde sadece implementation değişir, çağrı yerleri aynı kalır.

11.1 Sipariş -> Komisyon -> Coin Akışı

1. Kullanıcı ödeme yapar; payment-svc **payment.captured** olayını yayınlar (idempotency-key: order_id).
2. order-svc bu olayı tüketir, siparişi PAID durumuna alır ve **order.completed** olayını yayınlar.
3. commission-svc **order.completed**'i tüketir; kategori bazlı komisyon kuralını uygular ve bir accrual kaydı yazar.
4. Ay sonunda settlement job çalışır; satıcı bazında aylık toplam komisyonu hesaplar ve **commission.refund_requested** olayını yayınlar.
5. wallet-svc bu olayı tüketir, idempotency-key kontrolü yapar, double-entry ledger satırlarını atar.
6. Satıcı çekim talebi oluşturduğunda **withdraw.requested** olayı yayınlanır; treasury-svc banka transferini başlatır ve onay sonrası **withdraw.completed** yayınlar.

11.2 EventBus Interface'i (Go)

```
// internal/eventbus/bus.go
package eventbus

import "context"

type Event struct {
    Topic string
    EventID string
    OccurredAt time.Time
    Payload []byte
    Idempotency string
}

type Publisher interface {
    Publish(ctx context.Context, ev Event) error
}

type Handler func(ctx context.Context, ev Event) error

type Subscriber interface {
    Subscribe(topic string, group string, handler Handler) error
    Run(ctx context.Context) error
}

// MVP: Redis Streams implementation
// Scale-up: Kafka implementation, aynı interface'i karşılar
```

11.3 Telafi Stratejileri

- PSP capture başarısız: order-svc siparişi CANCELLED'a alır, rezerve edilen stok geri bırakılır.
- commission-svc'de kural uygulanamaz: olay DLQ'ya (Dead Letter Queue) düşer; manuel veya kuralı düzeltip yeniden işleme.
- wallet-svc ledger çift girdiyi atamadan önce kapanırsa: outbox tablosu sayesinde yeniden başlatma idempotent olur, çift yazım gerçekleşmez.

11.4 Olay Sözleşmeleri

```
// Stream: ecom.order.completed.v1
{
  "event_id": "uuid",
  "occurred_at": "RFC3339",
  "order_id": 123456,
  "user_id": 42,
  "items": [
    {"variant_id": 9001, "seller_id": 77, "qty": 2, "unit_price_minor": 19900}
  ],
  "currency": "TRY"
}

// Stream: fin.commission.refund_requested.v1
{
  "event_id": "uuid",
  "occurred_at": "RFC3339",
  "settlement_period": "2026-04",
  "seller_id": 77,
  "amount_minor": 12500,
  "reference": "STL-2026-04-77",
  "idempotency_key": "STL-2026-04-77"
}
```

12. API Tasarım Prensipleri

- Mobil tarafa REST/JSON; her endpoint OpenAPI 3 ile belgelenir, client SDK otomatik üretilir.
- Servisler arası HTTP/JSON (MVP); ileride gRPC + proto sözleşmeleri.
- Versiyonlama: URL path versiyonu (v1, v2); geçiş 12 ay desteklenir.
- Idempotency-Key header'ı: tüm yazma operasyonlarında zorunlu (özellikle finansal endpoint'lerde).
- Hata yapısı: RFC 7807 Problem Details + iç error_code (örn. ECOM_CART_OUT_OF_STOCK).
- Sayfalama: cursor-based; "page" parametresi yerine "after" cursor.
- Filtreleme: belgelenen, predictable bir sözdizimi; arbitrary query DSL yok.

12.1 Örnek Endpoint Sözleşmesi

```
POST /v1/orders/checkout
Headers:
Authorization: Bearer <jwt>
Idempotency-Key: <uuid>
X-App-Version: 1.4.2 (ios)
Body:
{
  "cart_id": "...",
  "address_id": 12,
  "payment_method_id": 33
}
Responses:
201 Created -> { "order_id": 123, "payment_url": "...", "status": "PENDING_PAYMENT" }
409 Conflict -> { "type": "/errors/cart-changed", "detail": "..." }
422 Unprocessable -> validation hataları listesi
```

13. Tek VDS Donanım Profili ve Kaynak Bütçesi

13.1 Donanım

- **CPU:** 6 vCPU (paylaşımlı veya dedicated, sağlayıcı politikasına göre)
- **RAM:** 24 GB
- **Disk:** 120 GB SSD (NVMe tercih)
- **Network:** 1 Gbps şarj edilmemiş bağlantı (sınırsız trafik)
- **Aylık ücret:** 940 TL

13.2 RAM Bütçesi (24 GB)

Servis	Konfig	RAM	Notlar
postgres-ecom	shared_buffers=2GB, effective_cache_size=6GB	4 GB	Ana DB
postgres-ledger	shared_buffers=1GB, effective_cache_size=3GB	2 GB	Ayrı container
redis	maxmemory=800mb, allkeys-lru	1 GB	Cache + sepet + Streams
meilisearch	default	1 GB	Arama
pgbouncer (x2)	min	100 MB	Connection pool
10-12 Go servisi	GOMEMLIMIT=200MiB her biri	2.5 GB	Toplam
caddy	min	100 MB	Reverse proxy
grafana-agent	min	300 MB	Logs/metrics
sentry-relay (opsiyonel)	min	200 MB	Self-hosted istemiyorsak SaaS
docker daemon + sistem	-	1 GB	Linux kernel + Docker
Linux page cache + buffer	kalan	~11 GB	Postgres performansı için kritik
TOPLAM kullanılan		~13 GB	Sürekli kullanım
TOPLAM cache + buffer		~11 GB	İşletim sistemi yararına

ÖNEMLİ

Linux page cache 11 GB'lık alanı boş bırakmaz; aktif kullanılan disk bloklarını otomatik cache'ler. Bu sayede Postgres'e gelen okuma sorgularının büyük kısmı disk'e değil RAM'e iner.

13.3 Disk Bütçesi (120 GB)

Kalem	Başlangıç	6 ay sonra	9 ay sonra
OS + Docker imajları	12 GB	15 GB	18 GB
postgres-ecom data	5 GB	30 GB	50 GB
postgres-ledger data	2 GB	8 GB	12 GB
Redis persistence (RDB+AOF)	2 GB	4 GB	5 GB
Meilisearch index	1 GB	3 GB	5 GB
Lokal log/metric (kısa retention)	5 GB	10 GB	12 GB
Lokal son backup snapshot	5 GB	8 GB	10 GB
Boş buffer	88 GB	42 GB	8 GB
TOPLAM kullanılan	32 GB	78 GB	112 GB

9. ay civarında disk kritik bölgeye girer. İkinci sunucuya geçiş bu noktadan önce planlanmalıdır. Tetikleyici: kullanılan disk %70'i geçtiğinde alarm.

13.4 CPU Bütçesi (6 vCPU)

- Ortalama yük (30K kullanıcı/saat = 300 RPS): %35-50 CPU
- Peak yük (push notif, kampanya): %70-85 CPU
- Burst kapasite: %15-30 yedek var
- Postgres VACUUM ve autoanalyze gece çalışır; gündüz CPU'ya minimal etki

13.5 Network Bütçesi

- Ortalama trafik: 30-80 Mbps (CDN sayesinde düşük)
- Peak: 200-300 Mbps
- 1 Gbps fiziksel limit yeterli; CloudFlare arkasında DDoS koruması mevcut

14. Docker Compose ile Tam Kurulum

Bu bölüm tek VDS üzerinde tüm sistemin nasıl ayağa kaldırıldığını adım adım gösterir. Tüm yapılandırmalar production-ready'dir; sadece scale-out (ölçek artırma) için Kubernetes geçişi gerektiğinde aynı container'lar oraya taşınır.

14.1 Dizin Yapısı

```

/opt/mopro/
├─ docker-compose.yml
├─ .env # tüm secret'lar (chmod 600)
├─ caddy/
│   └─ Caddyfile
├─ postgres-ecom/
│   └─ postgresql.conf
│       └─ init/ # ilk açılışta çalışan SQL'ler
├─ postgres-ledger/
│   └─ postgresql.conf
│       └─ init/
├─ redis/
│   └─ redis.conf
├─ pgbouncer/
│   └─ pgbouncer-ecom.ini
│       └─ pgbouncer-ledger.ini
├─ grafana-agent/
│   └─ agent.yaml
└─ data/ # named volume yerine bind mount

├─ postgres-ecom/
├─ postgres-ledger/
├─ redis/
└─ meilisearch/

/opt/mopro/backups/ # restic backup local cache
/var/log/mopro/ # docker compose logs symlink

```

14.2 docker-compose.yml (Çekirdek)

```

version: "3.9"

x-go-defaults: &go-defaults
restart: unless-stopped
logging:
driver: json-file
options:
max-size: "20m"
max-file: "5"
environment:
LOG_LEVEL: info
ENV: production
REDIS_URL: redis://redis:6379/0
EVENT_BUS: redis_streams
networks: [ mopro-net ]

services:
caddy:
image: caddy:2-alpine
restart: unless-stopped
ports:
- "80:80"
- "443:443"
volumes:
- ./caddy/Caddyfile:/etc/caddy/Caddyfile:ro
- caddy_data:/data
- caddy_config:/config
networks: [ mopro-net ]

postgres-ecom:
image: postgres:16-alpine
restart: unless-stopped
environment:
POSTGRES_DB: mopro_ecom
POSTGRES_USER: ecom_admin
POSTGRES_PASSWORD: ${ECOM_DB_PASSWORD}
volumes:
- ./data/postgres-ecom:/var/lib/postgresql/data
- ./postgres-ecom/init:/docker-entrypoint-initdb.d:ro
- ./postgres-ecom/postgresql.conf:/etc/postgresql/postgresql.conf:ro
command: postgres -c config_file=/etc/postgresql/postgresql.conf
networks: [ mopro-net ]
healthcheck:
test: ["CMD-SHELL", "pg_isready -U ecom_admin -d mopro_ecom"]

```



```

interval: 10s
timeout: 3s
retries: 5

postgres-ledger:
image: postgres:16-alpine
restart: unless-stopped
environment:
POSTGRES_DB: mopro_ledger
POSTGRES_USER: ledger_admin
POSTGRES_PASSWORD: ${LEDGER_DB_PASSWORD}
volumes:
- ./data/postgres-ledger:/var/lib/postgresql/data
- ./postgres-ledger/init:/docker-entrypoint-initdb.d:ro
- ./postgres-ledger/postgresql.conf:/etc/postgresql/postgresql.conf:ro
command: postgres -c config_file=/etc/postgresql/postgresql.conf
networks: [ mopro-fin-net ] # AYRI NETWORK, ecom servisleri buraya erişemez
healthcheck:
test: ["CMD-SHELL", "pg_isready -U ledger_admin -d mopro_ledger"]
interval: 10s
timeout: 3s
retries: 5

pgbouncer-ecom:
image: edoburu/pgbouncer:latest
restart: unless-stopped
environment:
DATABASE_URL:
"postgres://ecom_admin:${ECOM_DB_PASSWORD}@postgres-ecom:5432/mopro_ecom"
POOL_MODE: transaction
MAX_CLIENT_CONN: 500
DEFAULT_POOL_SIZE: 30
networks: [ mopro-net ]
depends_on: [ postgres-ecom ]

pgbouncer-ledger:
image: edoburu/pgbouncer:latest
restart: unless-stopped
environment:
DATABASE_URL:
"postgres://ledger_admin:${LEDGER_DB_PASSWORD}@postgres-ledger:5432/mopro_ledger"
POOL_MODE: transaction
MAX_CLIENT_CONN: 200
DEFAULT_POOL_SIZE: 20
networks: [ mopro-fin-net ]
depends_on: [ postgres-ledger ]

```

```

redis:
image: redis:7-alpine
restart: unless-stopped
command: ["redis-server", "/usr/local/etc/redis/redis.conf"]
volumes:
- ./redis/redis.conf:/usr/local/etc/redis/redis.conf:ro
- ./data/redis:/data
networks: [ mopro-net ]

meilisearch:
image: getmeili/meilisearch:v1.6
restart: unless-stopped
environment:
MEILI_ENV: production
MEILI_MASTER_KEY: ${MEILI_MASTER_KEY}
MEILI_NO_ANALYTICS: "true"
volumes:
- ./data/meilisearch:/meili_data
networks: [ mopro-net ]

identity-svc:
<<: *go-defaults
image: ghcr.io/mopro/identity-svc:1.0.0
environment:
DB_HOST: pgbouncer-ecom
DB_PORT: 5432
DB_NAME: mopro_ecom
DB_USER: identity_user
DB_PASSWORD: ${IDENTITY_DB_PASSWORD}
JWT_SIGNING_KEY: ${JWT_SIGNING_KEY}

catalog-svc:
<<: *go-defaults
image: ghcr.io/mopro/catalog-svc:1.0.0
environment:
DB_HOST: pgbouncer-ecom
MEILI_HOST: http://meilisearch:7700
MEILI_KEY: ${MEILI_MASTER_KEY}

cart-svc:
<<: *go-defaults
image: ghcr.io/mopro/cart-svc:1.0.0

order-svc:
<<: *go-defaults

```

```

image: ghcr.io/mopro/order-svc:1.0.0
environment:
DB_HOST: pgbouncer-ecom
PSP_API_KEY: ${PSP_API_KEY}

payment-svc:
<<: *go-defaults
image: ghcr.io/mopro/payment-svc:1.0.0

seller-svc:
<<: *go-defaults
image: ghcr.io/mopro/seller-svc:1.0.0

commission-svc:
<<: *go-defaults
image: ghcr.io/mopro/commission-svc:1.0.0
networks: [ mopro-net, mopro-fin-net ]

wallet-svc:
<<: *go-defaults
image: ghcr.io/mopro/wallet-svc:1.0.0
environment:
DB_HOST: pgbouncer-ledger
DB_PORT: 5432
DB_NAME: mopro_ledger
DB_USER: wallet_user
DB_PASSWORD: ${WALLET_DB_PASSWORD}
networks: [ mopro-fin-net ]

treasury-svc:
<<: *go-defaults
image: ghcr.io/mopro/treasury-svc:1.0.0
networks: [ mopro-fin-net ]

notification-svc:
<<: *go-defaults
image: ghcr.io/mopro/notification-svc:1.0.0
environment:
FCM_SERVER_KEY: ${FCM_SERVER_KEY}
SES_REGION: eu-central-1
SMS_PROVIDER_API: ${SMS_API_KEY}

grafana-agent:
image: grafana/agent:latest
restart: unless-stopped
volumes:

```

```
- ./grafana-agent/agent.yaml:/etc/agent.yaml:ro
- /var/run/docker.sock:/var/run/docker.sock:ro
- /var/log:/var/log:ro
command: ["-config.file=/etc/agent.yaml"]
networks: [ mopro-net ]

networks:
mopro-net:
driver: bridge
mopro-fin-net:
driver: bridge # FİNTECH İZOLASYONU - sadece commission-svc bridge yapar

volumes:
caddy_data:
caddy_config:
```

FİNTECH İZOLASYONU

Network izolasyonu: mopro-fin-net'e sadece wallet-svc, treasury-svc, postgres-ledger ve commission-svc bağlıdır. Diğer e-ticaret servisleri postgres-ledger'a TCP seviyesinde ulaşamaz. commission-svc, iki network'a da bağlı tek köprüdür ve sadece olay yayını/tüketimi yapar.

15. Caddy Reverse Proxy ve TLS

Caddy, otomatik Let's Encrypt sertifika yönetimi ile public trafiği iç servislere yönlendirir. Konfigürasyon basittir; bir Nginx kurulumunun karmaşıklığının onda biriyle TLS, HTTP/3, gzip, rate-limit ve route mantığı bir arada gelir.

15.1 Caddyfile

```
# /opt/mopro/caddy/Caddyfile
{
  email ops@moproshop.com
  admin off # Caddy admin API kapalı
  servers {
  protocols h1 h2 h3
  }
}

# Mobil API
api.moproshop.com {
  encode zstd gzip

  # Rate limit: kullanıcı başına 100 r/dk
  rate_limit {
    zone per_ip {
      key {remote_host}
      events 100
      window 1m
    }
  }

  # Identity rotaları (telefon, OTP) için sıkı limit
  @auth path /v1/auth/*
  handle @auth {
    rate_limit {
      zone auth_per_ip {
        key {remote_host}
        events 10
        window 1m
      }
    }
    reverse_proxy identity-svc:8080
  }

  # Cüzdan endpoint'leri için ekstra sıkı
  @wallet path /v1/wallet/* /v1/withdraw/*
  handle @wallet {
    rate_limit {
      zone wallet_per_ip {
        key {remote_host}
        events 30
        window 1m
      }
    }
  }
}
```

```

}
reverse_proxy wallet-svc:8080
}

# Diğer servis route'ları
@catalog path /v1/products/* /v1/categories/* /v1/search
handle @catalog { reverse_proxy catalog-svc:8080 }

@cart path /v1/cart/*
handle @cart { reverse_proxy cart-svc:8080 }

@orders path /v1/orders/*
handle @orders { reverse_proxy order-svc:8080 }

@support path /v1/support/* /v1/articles/*
handle @support { reverse_proxy support-svc:8080 }

# Default 404
handle { respond 404 }

log {
output file /var/log/caddy/api.log {
roll_size 100mb
roll_keep 7
}
format json
}
}

# Satıcı paneli (web)
seller.moproshop.com {
encode zstd gzip
reverse_proxy seller-panel-web:3000
}

# Public görsel servis (Backblaze B2 önünde cache)
img.moproshop.com {
reverse_proxy https://f000.backblazeb2.com/file/mopro-media {
header_up Host {upstream_hostport}
}
header Cache-Control "public, max-age=2592000"
}

```

15.2 Caddy'nin Kazandırdıkları

- Otomatik Let's Encrypt sertifikası — her domain için ayrı, otomatik yenilenir.

- HTTP/3 (QUIC) desteđi — mobilde yavaş bağlantılarda %15-30 daha hızlı.
- Built-in rate limiter — fail2ban veya Nginx Lua gerek yok.
- JSON formatlı access log — Grafana Agent direkt parse edebilir.
- Grace reload — config deđişikliđi aktif bağlantıyı kesmez.

16. PostgreSQL Dual Kurulum (Ecom + Ledger İzolasyonu)

16.1 İzolasyon Disiplinleri

- İki ayrı container, iki ayrı volume, iki ayrı port (5432 ve 5433).
- İki ayrı admin kullanıcı, iki ayrı şifre, iki ayrı .env değişkeni.
- Ledger DB **mopro-fin-net** network'una bağlı; e-ticaret servisleri ona ulaşamaz.
- Ledger DB'sine erişebilen tek servis: wallet-svc, treasury-svc, commission-svc (sadece olay yayını için).
- Backup dosyaları farklı parolalarla şifrelenir (restic için ayrı repository).

16.2 postgres-ecom postgresql.conf (Optimize)

```
# 24 GB RAM'li VDS'te postgres-ecom için
listen_addresses = '*'
port = 5432

# Memory
shared_buffers = 2GB
effective_cache_size = 6GB
work_mem = 16MB
maintenance_work_mem = 256MB

# WAL
wal_level = replica
max_wal_size = 2GB
min_wal_size = 256MB
checkpoint_completion_target = 0.9
checkpoint_timeout = 15min

# Connection
max_connections = 100 # PgBouncer arkasında
superuser_reserved_connections = 5

# Query planning
random_page_cost = 1.1 # SSD için
effective_io_concurrency = 200

# Logging
log_destination = 'stderr'
logging_collector = off # Docker log driver'a bırak
log_min_duration_statement = 200 # 200 ms üstü sorguları logla
log_checkpoints = on
log_connections = off
log_disconnections = off
log_lock_waits = on

# Autovacuum
autovacuum = on
autovacuum_max_workers = 3
autovacuum_naptime = 30s

# Replication preparedness (ileride read replica için)
wal_keep_size = 1GB
```

16.3 postgres-ledger postgresql.conf (Daha Sıkı)

```
# Ledger için daha agresif fsync, daha küçük bellek
listen_addresses = '*'
port = 5432 # container içinde aynı port

shared_buffers = 1GB
effective_cache_size = 3GB
work_mem = 8MB
maintenance_work_mem = 128MB

# Finansal veri = fsync mutlaka tam
fsync = on
synchronous_commit = on
full_page_writes = on
wal_compression = on

# Connection daha az (ledger trafiği düşük)
max_connections = 50

# Logging her şeyi
log_min_duration_statement = 100
log_statement = 'mod' # tüm INSERT/UPDATE/DELETE'i logla (audit için)
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0

# Append-only ledger için VACUUM agresif değil
autovacuum_vacuum_scale_factor = 0.1
```

16.4 Init Scripts

Postgres container'ı ilk açıldığında /docker-entrypoint-initdb.d/ içindeki .sql dosyaları sırayla çalışır. Her servisin tablo ve kullanıcısı burada tanımlanır.

```
-- /opt/mopro/postgres-ecom/init/01-users.sql
CREATE USER identity_user WITH PASSWORD 'changeme1';
CREATE USER catalog_user WITH PASSWORD 'changeme2';
CREATE USER order_user WITH PASSWORD 'changeme3';
-- ... (production'da .env'den okuyacak şekilde Helm-benzeri template ile üretilir)

CREATE SCHEMA identity AUTHORIZATION identity_user;
CREATE SCHEMA catalog AUTHORIZATION catalog_user;
CREATE SCHEMA orders AUTHORIZATION order_user;

-- Servis sadece kendi şemasına yazabilir
```

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
```

16.5 Migration Disiplini

- Her servis kendi migration'ını yönetir (golang-migrate, sqlc gibi araçlar).
- Migration'lar geriye dönük uyumlu olmak zorunda (expand-and-contract pattern).
- Önce şema değişikliği canlıya gider, sonra kod değişikliği — bu sıra rollback'i basitleştirir.
- Ledger DB'sinde DESTRUCTIVE migration (DROP TABLE, ALTER COLUMN type) **YASAK**; yeni kolon eklenir, eski deprecated olarak işaretlenir.

17. Redis, Meilisearch ve PgBouncer Konfigürasyonu

17.1 Redis

```
# /opt/mopro/redis/redis.conf
bind 0.0.0.0
protected-mode yes
port 6379
requirepass ${REDIS_PASSWORD}

# Persistence
appendonly yes
appendfsync everysec
save 900 1
save 300 10
save 60 10000

# Memory
maxmemory 800mb
maxmemory-policy allkeys-lru

# Streams trim (event bus için)
# her stream için XADD ile MAXLEN ~ 10000 verilir, böylece sınırsız büyüzmez

# Slow log
slowlog-log-slower-than 10000
slowlog-max-len 128

# Keyspace notifications (cache invalidation için)
notify-keyspace-events Ex
```

17.2 Meilisearch

- Türkçe destekli, 2-3 GB veriye kadar sıkıntısız.
- Indexing: catalog-svc her ürün değişikliğinde Meilisearch'e dokunur.
- Atomik update: tam değişim sıfırlama yapmaz; sadece güncellenir.
- Search latency: lokal disk'te 5-15 ms; Caddy üzerinden mobil için 50-80 ms toplam.
- Yedekleme: günlük dump /dumps klasörüne; restic ile B2'ye gider.

17.3 PgBouncer

- Postgres'in 'her bağlantı = ayrı process' modeli yüksek concurrency'de tükenir.
- PgBouncer transaction mode: 500 client bağlantısı, 30 gerçek Postgres bağlantısı.
- Servis kodu PgBouncer'a bağlanır, Postgres'i hiç bilmez.
- Hot standby planı: ilerideki Postgres replica'lara aynı PgBouncer route eder; servis kodu değişmez.

17.4 Connection Topolojisi

```
[Go servisi] --> [PgBouncer (transaction mode)] --> [Postgres]  
max=20 max=500 client / 30 server max=100
```

```
# 12 servis × 20 bağlantı = 240 client ihtiyaç
```

```
# PgBouncer 240'i alır ve sadece 30'unu Postgres'e açar
```

```
# Postgres rahat nefes alır, 70 bağlantı boş kalır
```

18. Backup Stratejisi

Tek VDS'te yaşanan en korkulu senaryo veri kaybıdır. Disk corruption, ransomware, yanlışlıkla DROP TABLE — hepsinin tek çözümü **dış lokasyonda yedektir**. Aynı VDS'in başka diskine yedek almak yedek değildir.

18.1 Hedefler

- **RPO (kabul edilen veri kaybı):** 1 saat (en kötü ihtimalle son saatlik veri kaybı).
- **RTO (kurtarma süresi):** 4 saat (yeni VDS + restore + DNS).
- **Yedek konumu:** Backblaze B2 (eu-central region) — VDS sağlayıcısından bağımsız.
- **Şifreleme:** Restic native AES-256; anahtar şirket sırrıdır, B2'de saklanmaz.
- **Retention:** günlük 30 gün, haftalık 12 hafta, aylık 12 ay.

18.2 Backup Mimarisi

```
[VDS]
cron @hourly -> wal-archive -> Backblaze B2 (postgres-ecom WAL)
cron @hourly -> wal-archive -> Backblaze B2 (postgres-ledger WAL)
cron @daily -> pg_dump -> restic backup -> Backblaze B2 (full snapshot)
cron @weekly -> meilisearch dump + redis RDB -> restic
cron @daily -> docker volume tarball -> restic
```

18.3 Backup Script (Bash)

```
#!/usr/bin/env bash
# /opt/mopro/scripts/backup.sh
# Günlük tam backup. Cron: 0 3 * * * /opt/mopro/scripts/backup.sh

set -euo pipefail
source /opt/mopro/.env

DATE=$(date +%Y%m%d_%H%M%S)
WORK=/opt/mopro/backups/$DATE
mkdir -p "$WORK"

# 1. Postgres dumps
docker exec postgres-ecom pg_dumpall -U ecom_admin \
| gzip > "$WORK/postgres-ecom.sql.gz"

docker exec postgres-ledger pg_dumpall -U ledger_admin \
| gzip > "$WORK/postgres-ledger.sql.gz"

# 2. Redis RDB snapshot
docker exec redis redis-cli -a "$REDIS_PASSWORD" SAVE
docker cp redis:/data/dump.rdb "$WORK/redis.rdb"

# 3. Meilisearch dump
curl -s -X POST -H "Authorization: Bearer $MEILI_MASTER_KEY" \
http://localhost:7700/dumps
sleep 30
docker cp meilisearch:/meili_data/dumps "$WORK/meilisearch_dumps"

# 4. Restic ile encrypted, deduplicated B2 push
export RESTIC_REPOSITORY="b2:mopro-backups:/full"
export RESTIC_PASSWORD="$RESTIC_PASSWORD"
export B2_ACCOUNT_ID="$B2_KEY_ID"
export B2_ACCOUNT_KEY="$B2_APP_KEY"

restic backup --tag full --tag "$DATE" "$WORK"

# 5. Lokal eski dosyaları temizle (yer kazanç için)
find /opt/mopro/backups -maxdepth 1 -type d -mtime +2 -exec rm -rf {} +

# 6. Restic retention policy
restic forget --keep-daily 30 --keep-weekly 12 --keep-monthly 12 --prune

# 7. Healthcheck ping
curl -sf https://hc-ping.com/$HEALTHCHECK_BACKUP_UUID || true
```


18.4 WAL Archiving (Saatlik)

Postgres'in WAL (Write-Ahead Log) dosyalarını saatlik B2'ye gönderirsek, point-in-time recovery (PITR) yapabiliriz. Yani "saat 14:23'teki haline getir" mümkün olur.

```
# postgres-ecom postgresql.conf'a ek
archive_mode = on
archive_command = '/opt/mopro/scripts/wal-push.sh %p %f'
archive_timeout = 300 # en az 5 dk'da bir, sıkıştırıp gönder

# wal-push.sh basitçe:
# restic backup --tag wal "$1"
# (veya wal-g, pgBackRest gibi profesyonel araçlar)
```

18.5 Restore Tatbikatı (HAFTALIK Zorunlu)

Yedeği aldığını ispat eden tek şey, yedeği geri yükleyebilmektir. Bu yüzden haftada bir kez (örneğin Pazar gecesi) bir restore tatbikatı çalıştırılır.

```
#!/usr/bin/env bash
# /opt/mopro/scripts/restore-drill.sh
# Haftalık otomatik tatbikat. Cron: 0 4 * * 0

set -euo pipefail

# 1. İkinci, geçici Postgres container ayağa kaldır (port 6432)
docker run -d --rm --name postgres-restore-test \
-e POSTGRES_PASSWORD=test \
-p 6432:5432 \
postgres:16-alpine

sleep 10

# 2. En son backup'ı çek
LATEST=$(restic snapshots --tag full --json | jq -r '[-1].id')
restic restore "$LATEST" --target /tmp/restore-test

# 3. Dump'ı yeni Postgres'e yükle
gunzip -c /tmp/restore-test/*/postgres-ecom.sql.gz \
| docker exec -i postgres-restore-test psql -U postgres

# 4. Bütünlük kontrolü: tablo sayısı ve önemli sayımlar
docker exec postgres-restore-test psql -U postgres -d mopro_ecom -c \
"SELECT count(*) FROM products"

# 5. Cleanup
docker stop postgres-restore-test
rm -rf /tmp/restore-test
```

```
# 6. Sonuç bildir (Slack webhook veya healthchecks.io)  
curl -sf https://hc-ping.com/$HEALTHCHECK_RESTORE_UUID
```

ZORUNLU DİSİPLİN

Backup script'i kurmak ve unutmak en sık yapılan hatadır. Bu yüzden HAFTALIK restore tatbikatı zorunlu olarak takvime konur. Tatbikat başarısız olursa ekibe alarm düşer ve sorun çözülene kadar başka iş yapılmaz.

19. Gözlemlenebilirlik

19.1 Stratejik Tercih

Tek VDS senaryosunda Prometheus + Loki + Tempo'yu lokalde çalıştırmak hem RAM (4 GB+) hem disk (~30 GB) yer; üstelik sunucu çökerse logları da kaybolur. Bu yüzden **Grafana Cloud free tier** kullanılır: 10K seri metrik, 50 GB log/14 gün retention, 50 GB trace ücretsiz.

19.2 Grafana Agent (Tek Toplayıcı)

```
# /opt/mopro/grafana-agent/agent.yaml
server:
log_level: info

metrics:
global:
scrape_interval: 30s
external_labels:
cluster: mopro-prod
configs:
- name: integrations
remote_write:
- url: https://prometheus-prod-01-eu-west-0.grafana.net/api/prom/push
basic_auth:
username: ${GRAFANA_PROM_USER}
password: ${GRAFANA_PROM_PASS}
scrape_configs:
# Docker container metrics
- job_name: docker
docker_sd_configs:
- host: unix:///var/run/docker.sock
relabel_configs:
- source_labels: [__meta_docker_container_label_com_docker_compose_service]
target_label: service
# Postgres metrics (postgres_exporter sidecar)
- job_name: postgres
static_configs:
- targets: ['postgres-exporter:9187']

logs:
configs:
- name: docker-logs
clients:
- url: https://logs-prod-eu-west-0.grafana.net/loki/api/v1/push
basic_auth:
username: ${GRAFANA_LOKI_USER}
password: ${GRAFANA_LOKI_PASS}
positions:
filename: /tmp/positions.yaml
scrape_configs:
- job_name: docker
docker_sd_configs:
- host: unix:///var/run/docker.sock
refresh_interval: 5s
```

```

relabel_configs:
- source_labels: [__meta_docker_container_name]
target_label: container

traces:
configs:
- name: tempo
remote_write:
- endpoint: tempo-eu-west-0.grafana.net:443
basic_auth:
username: ${GRAFANA_TEMPO_USER}
password: ${GRAFANA_TEMPO_PASS}
receivers:
otlp:
protocols:
grpc:
endpoint: 0.0.0.0:4317

```

19.3 SLO Tanımları

Servis	SLI	SLO Hedefi	Hata Bütçesi (30 gün)
api.moproshop.com	p95 latency	< 800 ms	0.1% indirgeme
order-svc /checkout	Başarı oranı	> %99.0	%1 hata bütçesi
wallet-svc balance	p95 latency	< 200 ms	0.05% indirgeme
catalog browse	Erişilebilirlik	%99.9	43 dk/ay
payment capture	Başarı oranı	> %98.5	%1.5

19.4 Alarm Hiyerarşisi

- **Tier 1 (telefon arama):** Ledger reconciliation farkı, ödeme PSP başarı %5'in altı, çekim 30 dk takılı.
- **Tier 2 (Slack + push):** p95 latency SLO ihlali, Postgres connection pool %85, disk %75.
- **Tier 3 (sadece dashboard):** Cache hit rate düşüşü, queue lag.
- Alarm aracı: Better Stack (Free tier) veya Grafana OnCall (free).

20. CloudFlare CDN, WAF ve DDoS Koruması

CloudFlare Free tier, küçük bir startup için neredeyse mucize gibidir: ücretsiz CDN, ücretsiz WAF, ücretsiz DDoS koruması, sınırsız trafik. Yapılması gereken sadece DNS'i CloudFlare'e taşımak ve bazı kuralları yapılandırmaktır.

20.1 Yapılandırma Adımları

1. Domain'i CloudFlare'a taşı (DNS).
2. api.moproshop.com → VDS public IP (proxy on, turuncu bulut).
3. img.moproshop.com → Backblaze B2 endpoint (proxy on).
4. seller.moproshop.com → VDS public IP.
5. SSL/TLS modu: **Full (strict)** — Caddy zaten Let's Encrypt veriyor.
6. Always Use HTTPS, HSTS aktif.
7. Browser Integrity Check aktif.
8. Bot Fight Mode aktif (free tier basic).

20.2 Cache Kuralları

Endpoint pattern	Cache TTL	Notlar
/v1/products/{id}	60 sn	Edge cache; ürün değişince purge
/v1/categories	5 dk	Yavaş değişen veri
/v1/banners	2 dk	Anasayfa banner'ları
/v1/search	Bypass	Kişiselleştirilmiş, cache'lenmez
/v1/cart/*	Bypass	Per-user
/v1/wallet/*	Bypass	Finansal, asla cache değil
/v1/orders/*	Bypass	Per-user, dinamik
img.moproshop.com/*	30 gün	Görseller agresif cache

20.3 WAF Kuralları

- OWASP Core Rule Set Free tier'da basic seviyede aktif.
- Custom rule: /v1/auth/* path'ine dakikada 10'dan fazla istek = challenge (CAPTCHA).
- Custom rule: bilinmeyen User-Agent + POST = challenge.
- Custom rule: kabul edilmeyen ülkelerden gelen trafik (örn. botnet kümeleri) = block.
- Rate Limit: tüm domain için IP başına 600 istek/dk üstü = block 10 dk.

20.4 DDoS Koruması

- Free tier'da L3/L4 DDoS koruması otomatik (sınırsız).
- L7 (HTTP) DDoS için "Under Attack Mode" tek tıkla aktive edilebilir; tüm gelen istekler 5 sn JavaScript challenge'a tabi.
- VDS'in gerçek IP'si CloudFlare arkasında saklı; bot saldırıları VDS'e ulaşmaz.

21. Dağıtık Mimariye Hazırlık (Distributed-Ready Disiplinler)

Bu bölüm, tek VDS'te çalışırken yarın 2-3 VDS'e veya Kubernetes cluster'ına sıfır kod değişikliğiyle taşıyabilmemizi sağlayan 12 disiplini listeler. Her biri ilk gün uygulanır.

21.1 12 Disiplin

- 1. Container DNS isimleri:** localhost yok; postgres-ecom, redis gibi DNS isimleri.
- 2. Env değişkeni ile bağlantı:** DB_HOST=\${DB_HOST}; kod hiçbir yerde sabit IP/host yazmaz.
- 3. Stateless servis kuralı:** Servis lokal disk'e dosya yazmaz; her şey DB veya object storage.
- 4. EventBus interface'i:** Redis Streams bugün, Kafka yarın; servis kodu farkı görmez.
- 5. Repository pattern:** DB erişimi interface arkasında; ileride read replica eklemek tek satır config.
- 6. Idempotency-key:** Her yazma operasyonunda zorunlu; retry sırasında çift yazım olmaz.
- 7. Migrationlar geriye dönük uyumlu:** expand-and-contract; eski sürüm yeni şemada da çalışır.
- 8. Sıfır implicit ortam bağımlılığı:** Servis 12-Factor; config tamamen env'den.
- 9. Health endpoint disiplini:** /healthz/live, /healthz/ready, /metrics ayrı portlarda.
- 10. Graceful shutdown:** SIGTERM alınca yeni istek almayı kes, mevcutları bitir, DB bağlantılarını kapat.
- 11. Outbox pattern (finansal olaylar):** DB transaction içinde event yazılır; ayrı bir worker stream'e push eder.
- 12. Servis log kontratı:** JSON, common alanlar (trace_id, service, version), PII yok.

21.2 İdempotency-Key Örnek Implementation

```
// internal/middleware/idempotency.go
func IdempotencyMiddleware(redis *redis.Client) gin.HandlerFunc {
    return func(c *gin.Context) {
        if c.Request.Method != "POST" && c.Request.Method != "PUT" {
            c.Next()
            return
        }
        key := c.GetHeader("Idempotency-Key")
        if key == "" {
            c.JSON(400, gin.H{"error": "Idempotency-Key header required"})
            c.Abort()
            return
        }
        cacheKey := fmt.Sprintf("idem:%s:%s", c.Request.URL.Path, key)
        if cached, err := redis.Get(c, cacheKey).Bytes(); err == nil {
            // Daha önce işlenmiş, aynı response'u dön
            c.Data(200, "application/json", cached)
            c.Abort()
            return
        }

        rec := httptest.NewRecorder()
        // ... handler'ı çalıştır, response'u kaydet, redis'e yaz (24 saat TTL)
        redis.Set(c, cacheKey, rec.Body.Bytes(), 24*time.Hour)
        c.Writer.Write(rec.Body.Bytes())
    }
}
```


21.3 Outbox Pattern (Finansal Olaylar İçin)

```
-- Ledger DB içinde bir outbox tablosu
CREATE TABLE outbox (
  id BIGSERIAL PRIMARY KEY,
  aggregate TEXT NOT NULL,
  event_type TEXT NOT NULL,
  payload JSONB NOT NULL,
  idempotency_key TEXT UNIQUE NOT NULL,
  published_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX outbox_unpublished ON outbox(created_at) WHERE published_at IS NULL;

-- Service kodu
BEGIN;
INSERT INTO ledger_entries (...);
INSERT INTO ledger_entries (...);
INSERT INTO outbox(aggregate, event_type, payload, idempotency_key)
VALUES ('wallet', 'commission.refund.posted', '{...}', 'STL-2026-04-77');
COMMIT;

-- Ayrı bir worker outbox'tan sürekli çeker, EventBus'a push eder, published_at set
eder.
-- Bu sayede DB yazımı ve event yayını ATOMIC olur.
```

22. İkinci Sunucuya Geçiş Planı

22.1 Tetikleyiciler

İkinci sunucu lüks değildir; aşağıdaki herhangi biri tetiklendiğinde 14 gün içinde tamamlanır.

- Disk kullanımı %70 (yaklaşık 84 GB) seviyesini geçti.
- Sürekli RAM kullanımı %85 üstüne çıktı (Linux page cache azaldı).
- 30K kullanıcı/saat seviyesi ardışık 7 gün boyunca aşıldı.
- Reklam yatırımı veya basın çıkışı 14 gün içinde planlanıyor.
- Aktif satıcı sayısı 100'ü geçti (FinTech tarafı kritikleşti).

22.2 Hedef Topoloji (2 Sunucu)

```
[VDS-1: app + ecom DB]
- caddy
- identity, catalog, cart, order, payment, seller, search, support, notification, media
- postgres-ecom + pgbouncer-ecom
- redis (e-ticaret cache + sepet)
- meilisearch
- grafana-agent

[VDS-2: fin (FinTech izolasyonu)]
- wallet-svc, treasury-svc
- postgres-ledger + pgbouncer-ledger
- commission-svc (event köprüsü)
- kendi redis (event bus partition için, opsiyonel)
- grafana-agent
- kendi backup zinciri (ayrı encryption key)

[CloudFlare]
- api.moproshop.com -> VDS-1
- VDS-1 ve VDS-2 arası WireGuard private VPN üzerinden konuşur
```

22.3 Adım Adım Geçiş (Önerilen Bakım Penceresi: Pazar 03:00-07:00)

- 1. T-7 gün:** VDS-2'yi sağlayıcıdan al, sıfırdan kur (Docker, restic, Grafana Agent).
- 2. T-3 gün:** VDS-1 ve VDS-2 arasında WireGuard tüneli kur, iki sunucu arası 10.0.0.0/24 private network.
- 3. T-1 gün:** VDS-2'de boş postgres-ledger container'ı ayağa kaldır, replication kullanıcısı oluştur.
- 4. T:** Bakım modu başlar; mobilde "kısa bakım" banner'ı.
- 5. T+5 dk:** postgres-ledger'dan pg_basebackup ile VDS-2'ye full snapshot.
- 6. T+30 dk:** Streaming replication başlat; VDS-2 hot standby'a geçer.
- 7. T+60 dk:** wallet-svc, treasury-svc'yi VDS-1'den durdur, VDS-2'de başlat (DB_HOST artık 10.0.0.2).

- 8. T+90 dk:** Smoke test: cüzdan endpoint'leri, çekim akışı, ledger reconcile job.
- 9. T+120 dk:** Bakım modu kapanır; trafik VDS-2 üzerinden cüzdan akışlarını kullanır.
- 10. T+24 saat:** Eski postgres-ledger volume'ünü VDS-1'den sil, disk yer kazan.

22.4 Üç Sunucu Topolojisi (Reklam Öncesi)

- VDS-1: app servisleri + caddy
- VDS-2: postgres-ecom + redis + meilisearch
- VDS-3: fin (postgres-ledger + wallet/treasury)
- Avantaj: app node bağımsız ölçeklenir; DB bağımsız büyür; FinTech tam izole.
- Yük dağılımı: caddy backend olarak app servisleri için VDS-1'in private IP'sini gösterir.

22.5 Geçiş Risklerini Azaltma

- Geçiş öncesi mobil sürümünde "bakım modu" UI'ı bulunsun (force-update'siz devreye alınabilen uzaktan flag).
- Geçiş öncesi gece son backup'ın tatbikatla başarılı olduğu doğrulanır.
- Rollback planı yazılı: VDS-2 başarısız olursa wallet-svc tekrar VDS-1'de başlatılır, DB_HOST eski değere döner.
- Geçiş sırasında bir SRE + bir backend lider + bir mobil developer aktif.

23. Scale-Up: Kubernetes ve Production-Grade Süreçler

Bu bölüm **ileride** aktive olacak süreçleri tarif eder. 200K+ kullanıcı/saat veya çok bölgesi operasyon noktasında VDS topolojisi yetmemeye başladığında devreye girer. Bugün uygulanmaz, ama bilmek mimari kararları bugünden doğru almayı sağlar.

23.1 Production-Grade Container İmajları

- Multi-stage Dockerfile: build aşaması Alpine + golang, runtime aşaması distroless/static.
- Non-root user (UID 65532), readOnlyRootFilesystem.
- Multi-arch: linux/amd64 + linux/arm64 (Graviton tasarrufu).
- SBOM (CycloneDX), keyless cosign imzalama.
- Trivy + Gype scan; HIGH/CRITICAL açık varsa pipeline kırılır.
- Manifestlerde imaj her zaman digest pin'li (sha256), tag yok.

23.2 Production Dockerfile (Go Servis)

```
# syntax=docker/dockerfile:1.7
ARG GO_VERSION=1.22
ARG SERVICE=order-svc

FROM golang:${GO_VERSION}-alpine AS builder
ARG SERVICE
ARG VCS_REF
ARG BUILD_DATE
WORKDIR /src

COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download

COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
  --mount=type=cache,target=/go/pkg/mod \
  CGO_ENABLED=0 GOOS=linux GOARCH=${TARGETARCH:-amd64} \
  go build -trimpath \
  -ldflags="-s -w" \
  -X main.version=${VCS_REF} \
  -X main.buildDate=${BUILD_DATE} \
  -o /out/app ./cmd/${SERVICE}

FROM gcr.io/distroless/static-debian12:nonroot AS runtime
ARG SERVICE
LABEL org.opencontainers.image.title="${SERVICE}" \
  org.opencontainers.image.vendor="Mopro" \
  org.opencontainers.image.source="https://github.com/mopro/platform"
USER nonroot:nonroot
WORKDIR /app
COPY --from=builder /out/app /app/app
EXPOSE 8080 9090
ENTRYPOINT ["/app/app"]
```

23.3 Kubernetes Deployment Manifesti (Sample)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-svc
  namespace: ecom
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
  selector: { matchLabels: { app: order-svc } }
  template:
    metadata:
      labels: { app: order-svc, domain: ecom }
    spec:
      serviceAccountName: order-svc
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
      seccompProfile: { type: RuntimeDefault }
      topologySpreadConstraints:
        - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector: { matchLabels: { app: order-svc } }
      containers:
        - name: app
          image: registry/mopro/order-svc@sha256:abc...
          ports:
            - { name: http, containerPort: 8080 }
            - { name: metrics, containerPort: 9090 }
          envFrom: [ { secretRef: { name: order-svc-secrets } } ]
          resources:
            requests: { cpu: "200m", memory: "256Mi" }
            limits: { cpu: "1000m", memory: "512Mi" }
          readinessProbe:
            httpGet: { path: /healthz/ready, port: metrics }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /healthz/live, port: metrics }
            periodSeconds: 10
          lifecycle:
            preStop:
              exec: { command: ["/app/app", "drain", "--timeout=25s"] }

```

```
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: { drop: ["ALL"] }
  volumeMounts:
    - { name: tmp, mountPath: /tmp }
  volumes:
    - { name: tmp, emptyDir: {} }
  terminationGracePeriodSeconds: 30
```

23.4 GitOps ile Deploy (ArgoCD)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: order-svc-prod, namespace: argocd }
spec:
  project: ecom
  source:
    repoURL: git@github.com:mopro/k8s-manifests.git
    targetRevision: main
  path: apps/ecom/order-svc/overlays/prod
  destination:
    server: https://prod-eks.api
    namespace: ecom
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

23.5 Scale-Up Tetikleyicileri

- 200K+ kullanıcı/saat sürekli yük (3 sunuculu yapı zorlanır).
- Multi-region operasyon (gecikme veya regülasyon).
- Ekibin DevOps/SRE kapasitesi 1+ kişiye çıktı.
- Float yönetimi nedeniyle daha yüksek availability SLO talebi.

24. Güvenlik Mimarisi

24.1 Kimlik ve Yetkilendirme

- Mobil OAuth2/OIDC + cihaz binding'i (refresh token cihaza bağlı).
- JWT 15 dk; refresh token 30 gün.
- RBAC: alıcı, satıcı, satıcı_personel, admin rolleri.
- Cüzdan çekimi: step-up authentication (biyometri/OTP).

24.2 Veri Güvenliği

- Transit'te TLS 1.3 (Caddy otomatik); ileride servisler arası mTLS (Istio).
- Diskte VDS sağlayıcı seviyesi şifreleme.
- PII alanları (TC, telefon, e-posta) uygulama seviyesinde encrypted (AES-GCM, anahtar .env).
- Loglara PII redaction filter; hash'lenmiş anahtarla join yapılır.
- Audit log immutable: B2 Object Lock veya append-only Postgres tablo.

24.3 KVKK Uyumu

- Aydınlatma metni ve açık rıza akışı uygulamada belgelenmiştir.
- Veri envanteri (VERBİS) güncel; her servisin işlediği PII kategorisi belgelenir.
- Silme talepleri: tombstone + zamanlı temizlik; ledger silinmez, kişisel veri ile bağı koparılır.
- Veri ihlali müdahale planı: 72 saat içinde KVKK bildirimine hazırlık.

24.4 Uygulama Güvenliği

- Edge'de CloudFlare WAF + Bot Fight Mode.
- Caddy seviyesinde rate-limit (kullanıcı + IP başına).
- Captcha veya cihaz risk skoru ile bot/abuse tespiti.
- Dependency tarama: govulncheck CI'da kırıncı eşik.
- Yıllık bağımsız penetrasyon testi (özellikle wallet-svc, treasury-svc).

24.5 Sırların Yönetimi

- MVP: /opt/mopro/.env dosyası; chmod 600, sadece root.
- Scale-up: HashiCorp Vault veya AWS Secrets Manager.
- Tüm secret'lar 90 günde bir rotate edilir (özellikle JWT signing key, DB password, PSP API key).
- Git'e commit edilmiş secret olduğu anda repo geçmişinden temizlenmez; secret döner.

25. CI/CD ve Deploy

25.1 Pipeline Akışı

```
[Developer PR] -> [GitHub Actions]
|
+-- lint (golangci-lint), vet, govulncheck
+-- unit + integration test
+-- docker buildx build (amd64)
+-- trivy scan (HIGH/CRIT block)
+-- ghcr.io'ya push (PR tag)
|
v
[Merge to main] -> [Release Workflow]
+-- semver tag
+-- production image push (ghcr.io/mopro/X:1.x.y)
+-- VDS'e deploy script tetikle
|
v
[Deploy script (VDS üzerinde SSH)]
docker compose pull
docker compose up -d <changed-service>
smoke test (curl /healthz/ready)
rollback hook (önceki imaj tag'i)
```

25.2 GitHub Actions Workflow Örneği

```
# .github/workflows/release.yml
name: release
on:
  push:
  tags: ['v*']

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
      id-token: write
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          file: build/Dockerfile
          build-args: |
            SERVICE=${ matrix.service }
            VCS_REF=${ github.sha }
            BUILD_DATE=${ steps.date.outputs.date }
          tags: |
            ghcr.io/mopro/${ matrix.service }:${ github.ref_name }
            ghcr.io/mopro/${ matrix.service }:latest

      - name: Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ghcr.io/mopro/${ matrix.service }:${ github.ref_name }
          severity: HIGH,CRITICAL
          exit-code: 1
```

```
deploy:
needs: build
runs-on: ubuntu-latest
steps:
- name: SSH deploy
uses: appleboy/ssh-action@v1
with:
host: ${ secrets.VDS_HOST }
username: deploy
key: ${ secrets.VDS_SSH_KEY }
script: |
cd /opt/mopro
export TAG=${ github.ref_name }
./scripts/deploy.sh $TAG
```

25.3 Rollback Stratejisi

- Her release için önceki imaj tag'i /opt/mopro/.previous-tag'e yazılır.
- Rollback: ./scripts/deploy.sh \$(cat .previous-tag)
- Smoke test başarısız olursa deploy.sh otomatik rollback yapar.
- DB migration bekleyen rollback'lerde önce app rollback yapılır, migration 'forward-only' kalır.

26. Maliyet Analizi

26.1 Sabit Aylık Maliyet (MVP)

Kalem	Aylık	Notlar
VDS (6c/24GB/120GB)	940 TL	Birincil sunucu
Backblaze B2 (backup, ilk 6 ay)	100-150 TL	~10-30 GB; sonra büyür
Domain (.com, yıllık \$15)	~40 TL	Aylık eşdeğeri
Apple Developer (yıllık \$99)	~110 TL	iOS yayını için zorunlu
Transactional email (SES)	~30 TL	OTP yedek + sipariş onayı
CloudFlare Free	0 TL	CDN + WAF + DDoS
Grafana Cloud Free	0 TL	Logs/metrics/traces
Sentry Free	0 TL	Error tracking
GitHub Free / Pro	0-150 TL	Container Registry + Actions free tier
TOPLAM SABİT	~1.220-1.420 TL	PSP/SMS hariç

26.2 İşlem Bazlı Maliyetler

Kalem	Birim Maliyet	Aylık Tahmin (örnek hacim)
PSP komisyonu	%1.5-2.5 (işleme göre)	1M TL hacim = 15-25K TL
SMS OTP	30-50 kuruş/SMS	1.000 OTP = ~400 TL
Push (FCM/APNs)	Ücretsiz	Sınırsız
KYC doğrulama (opsiyonel)	5-15 TL/satıcı	100 satıcı/ay = ~1.000 TL

26.3 İkinci Sunucuya Geçiş Sonrası

Kalem	MVP	2 Sunuculu	3 Sunuculu (reklam)
Birincil VDS	940 TL	940 TL	940 TL
İkinci VDS (fin)	-	~400 TL (4c/8GB)	~400 TL
Üçüncü VDS (DB)	-	-	~700 TL (4c/16GB)
Backup büyümesi	150 TL	250 TL	400 TL
CloudFlare Pro (ops?)	-	-	~700 TL
TOPLAM	~1.090 TL	~1.590 TL	~3.140 TL

ÖNEMLİ

Reklam dönemi öncesi maliyet artışı, reklam yatırımının kendisi yanında çok küçüktür. 100K TL reklam bütçesi vereceksen 1.500 TL ek altyapı yatırımı yapmaktan kaçınmak ekonomik değildir.

26.4 12 Aylık Senaryo

Ay	Setup	Aylık Sabit	Aylık (PSP+SMS tahmini)	Toplam dahil
1-3 (kapalı beta)	Tek VDS	1.250 TL	1.500-2.000 TL	
4-6 (soft launch)	Tek VDS + ek backup	1.350 TL	3.000-5.000 TL	
7-9 (organik büyüme)	2 VDS	1.700 TL	8.000-15.000 TL	
10-12 (reklam başlangıç)	3 VDS + CloudFlare Pro	3.200 TL	30.000-60.000 TL	

27. Geliştirme Yol Haritası

Faz	Süre	Modüller / Kapsam	Çıktı / Başarı Kriteri
Faz 0 - Temel	3-4 hafta	Repo, CI, VDS kurulum, Docker Compose iskeleti, identity-svc, tasarım sistemi	Geliştiriciler tek komutla local stack ayağa kaldırabiliyor
Faz 1 - E-ticaret Core	8-10 hafta	catalog, cart, order, payment, seller paneli, mobil temel akışlar	Kapalı beta: 5 satıcı, gerçek ödeme
Faz 2 - FinTech Core	6-8 hafta	wallet + ledger, commission, treasury, mobil cüzdan, çekim akışı	Ledger reconcile=0; ilk komisyon iade canlıda
Faz 3 - Deneyim	5-6 hafta	size-finder, support, bildirim, kupon, iade, AI yardımcı	İade %15+ düşüş; self-servis %40+
Faz 4 - Public Launch	4 hafta	Yük testi, sızma testi, KVKK, store yayını, restore tatbikatları	30K kullanıcı/saat yük testi geçti; ledger denetimi temiz
Faz 5 - 2. Sunucu	2 hafta	Fin VDS ayrımı, WireGuard, replication, smoke test	Sıfır-downtime geçiş; cüzdan VDS-2'de aktif
Faz 6 - Reklam Hazırlığı	2 hafta	3. sunucu, CloudFlare Pro, feature flag, runbook'lar	Reklam başlamadan SLO'lar onaylı

27.1 Önerilen Ekip

- 1 Teknik Lider / Mimar (yarı zaman backend kodlama)
- 2 Flutter mobil geliştirici
- 2-3 Go backend geliştirici (1'i wallet/ledger uzmanı)
- 1 Ürün Tasarımcı (UX/UI)
- 1 Ürün Yöneticisi (PM)
- Kısmi: QA otomasyon, FinTech avukat, mali müşavir
- MVP'de DevOps yarı zamanlı; Faz 5 sonrası tam zamanlıya çıkar.

28. KPI'lar ve Başarı Kriterleri

28.1 Teknik KPI'lar

- P95 anasayfa < 800 ms; P95 ödeme adımı < 1.5 sn.
- Servis SLO uyumu haftalık $\geq$ %99.9 (kritik servisler).
- Ledger reconciliation farkı = 0 (her gün).
- Çekim tamamlama süresi p95 < 24 saat.
- Restore tatbikatı haftalık başarılı.
- Critical CVE -> patch cycle < 7 gün.

28.2 Ürün KPI'ları

- Sepetten ödemeye dönüşüm > %35.
- Satıcı 90 günlük tutma > %80.
- Beden Bulucu kullanıcılarda iade %15+ düşüş.
- Self-servis destek çözüm oranı > %40.

28.3 Finansal KPI'lar

- Float büyüklüğü; aylık ortalama ve volatilité.
- Float'tan getiri marjı (net).
- Cüzdan -> nakde çekim oranı.
- Satıcı başına aylık ortalama komisyon iadesi.

29. Risk Analizi ve Azaltma Stratejileri

29.1 Birincil Riskler

Risk	Etkisi	Azaltma
Cüzdan & ledger bütünlüğü	Hatalı bakiye = doğrudan finansal kayıp ve güven krizi.	Append-only ledger; idempotency-key + outbox; ayrı DB; gece reconciliation; ayrı network.
Tek VDS donanım hatası	Tüm sistem çöker; veri kaybı ihtimali.	Saatlik WAL + günlük full backup B2'ye; haftalık restore tatbikatı; 4 saat RTO.
Performans / trafik spike	Reklamsız 30K/saat tamam; reklam basıldığında tek VDS yeter olmayabilir.	CDN + Redis cache; HPA hazır olmasa da feature flag ile pahalı özellikler kapatılır; reklam öncesi 2-3 sunuculu mimariye geçiş zorunlu.
Regülasyon & float	BDDK/MASAK e-para olarak yorumlayabilir.	FinTech avukat görüşü; emanet (escrow) hesap; KYC/AML modülü; bağımsız mali denetim.

29.2 İkincil Riskler

- PSP bağımlılığı: tek sağlayıcıya kalmak; en az 2 PSP entegrasyonu ve fail-over.
- Soğuk başlangıç: az satıcı = az ürün; lansman öncesi 50+ aktif satıcı önkaydı.
- İade dolandırıcılığı: beden bahanesiyle istismar; iade desen analizi.
- Mobil store onayı: kapalı devre coin'in "sanal para" olarak değerlendirilmemesi için dilin (loyalty/cashback) net olması.
- Disk dolması: 9. ay civarı; ikinci VDS planlanmazsa kritik.
- Tek admin bağımlılığı: VDS erişimi tek kişide olursa otobüs faktörü; en az 2 kişi sudo erişimi.

29.3 Lansman Öncesi Zorunlu Doğrulamalar

1. Restore tatbikatı 4 hafta üst üste başarılı.
2. Ledger reconciliation job 30 gün boyunca fark üretmedi.
3. Bağımsız penetrasyon testi (özellikle wallet-svc).
4. Yük testi: 30K kullanıcı/saat (~300 RPS) sürekli, ardından 1500 RPS burst.
5. FinTech avukat görüşü ve KVKK aydınlatma + açık rıza onayı.
6. App Store / Play Store policy review.
7. İkinci sunucu hazır (geçiş runbook'u yazılı, en az 1 kez test edilmiş).

30. Operasyonel Süreçler

30.1 On-Call ve Incident

- Better Stack (free) + Slack + sade bir status page.
- Tier 1 alarm: ledger fark, ödeme PSP %5'in altı, çekim takılı, disk %85.
- On-call rotation: 2 kişi haftalık değişimli (geliştirici + lider).
- Incident sınıflandırması: SEV1 (kullanıcı parası etkilenir), SEV2 (büyük yetenek kaybı), SEV3 (kısmi).
- Her SEV1/SEV2 sonrası 5 iş günü içinde public post-mortem (blameless).

30.2 Runbook Örneği - Çekim Takılı

1. Tetikleyici: çekim 30 dk'dan uzun bekleyen sayısı > 5
2. Kontrol komutları:


```
docker exec postgres-ledger psql -U ledger_admin -d mopro_ledger -c \
"SELECT count(*) FROM withdrawal_requests WHERE status='PENDING' AND created_at <
now() - interval '30 minutes'"
docker logs --tail 200 treasury-svc | grep ERROR
docker logs --tail 200 wallet-svc | grep ERROR
```
3. Hızlı eylemler:
 - Bank API down ise status page'e incident, mobil banner
 - Servis hatasında: docker compose up -d wallet-svc treasury-svc (yeniden başlat)
 - Hâlâ takılıysa: önceki imaj tag'iyle rollback
 - Manuel processing: ./scripts/treasury-replay.sh <withdrawal_id>
4. Kapanış: ledger reconcile job tetikle, post-mortem aç.

30.3 Bakım Penceresi Disiplini

- Planlı bakım: her ayın ikinci pazarı 03:00-05:00.
- Bakım öncesi mobilde "yakında bakım" banner'ı (24 saat öncesinden).
- Kritik bakımlar (Postgres major upgrade gibi) sadece bu pencerede.
- Hot fix bakımları feature flag ile downtime'sız.

31. Production Hardening Eki (Sürüm 3.1)

Bu bölüm, sürüm 3.0 dökümanının bağımsız bir teknik inceleme sürecinden sonra eklenen kritik hardening önlemlerini içerir. Hiçbir mimari karar değişmemiştir; tek VDS gerçekliğinde "küçük hataların büyük maliyete dönüşmesini" engelleyen operasyonel disiplinler eklenmiştir.

Eleştiri sürecinde dört ana nokta tespit edildi: (a) container resource limit'lerinin tanımlı olmaması, (b) container ve kernel hardening flag'lerinin eksik olması, (c) disk doluluk yönetiminin iyimser tahminlere dayalı olması, (d) outbox/saga'nın debug edilebilirliği için tracing ve replay aracının netleşmemiş olması. Aşağıda her biri için somut fix verilmiştir.

31.1 Container Resource Limit'leri ve OOM Koruması

Tek VDS'te en gerçekçi felaket senaryosu şudur: Bir Go servisi bellek sızıntısı yapar veya Meilisearch anlık olarak 5 GB ister; Linux kernel **OOM Killer**'ı devreye sokar; en çok RAM tüketen süreci öldürür - ki bu büyük ihtimalle Postgres'tir. Postgres öldüğü anda tüm sistem ayakta görünmeyecek hale gelir, geri gelirken WAL recovery yapar ve veri tutarsızlığı riski doğar.

Bu senaryoyu engellemenin tek yolu her container için kesin **mem_limit**, **cpus** ve **pids_limit** tanımlamaktır. Limit ihlali olduğunda Linux sadece o container'ı öldürür; restart policy yeniden başlatır; başka servisler etkilenmez.

Güncellenmiş Compose Şablonu

```
# Bölüm 14'teki x-go-defaults bloğuna eklenir
x-go-defaults: &go-defaults
restart: unless-stopped
mem_limit: 384m # OOM ihlalinde sadece bu container ölür
mem_reservation: 192m # Soft guarantee
cpus: '0.5'
pids_limit: 256 # Fork bombe koruması
security_opt:
  - no-new-privileges:true # Privilege escalation engellenir
cap_drop: [ALL] # Linux capability'leri sıfırlanır
read_only: true # Root filesystem yazılamaz
tmpfs:
  - /tmp:size=64M,mode=1777 # Geçici dosyalar bellek üstünde
ulimits:
  nofile: { soft: 4096, hard: 8192 }
logging:
  driver: json-file
  options: { max-size: "20m", max-file: "5" }
networks: [ mopro-net ]

# Veritabanı ve infrastructure servislerine özel limitler
postgres-ecom:
  mem_limit: 5g
  mem_reservation: 3g
  cpus: '2.0'
  shm_size: 256m
  read_only: false # Postgres yazma gerektirir; data volume'ünde

postgres-ledger:
  mem_limit: 3g
  mem_reservation: 2g
  cpus: '1.5'
  shm_size: 128m

redis:
  mem_limit: 1.2g # redis.conf maxmemory'sinden büyük olmalı
  cpus: '1.0'

meilisearch:
  mem_limit: 1.5g
  cpus: '1.0'

caddy:
  mem_limit: 256m
```

cpus: '0.5'

RAM Bütçesinin Yeniden Hesabı

Servis	mem_limit	Toplam Tahsis
postgres-ecom	5g	5 GB
postgres-ledger	3g	3 GB
redis	1.2g	1.2 GB
meilisearch	1.5g	1.5 GB
10-12 Go servisi × 384m	0.384g	~4.6 GB
caddy + grafana-agent + pgbouncer × 2	~0.7g	~0.7 GB
TOPLAM HARD LIMIT		~16 GB
OS + Docker daemon + page cache		~8 GB (kalan)

Bu hesap ile **tüm servisler limit'i aşıya bile toplam 16 GB'da durur**; Postgres ve Linux page cache için ayrılan 8 GB asla ihlal edilmez. Bu, OOM Killer'ın Postgres'i hedef seçmesini matematiksel olarak imkansız hale getirir.

DOĞRULAMA

Limit'leri ekledikten sonra üretim öncesi docker stats ile gerçek kullanımı 24 saat boyunca izle, limit'lerin gerçekliğini doğrula. Bir Go servisi sürekli %90+ mem_limit kullanıyorsa limit yetersizdir; trafiği analiz et veya limiti artır.

31.2 Kernel ve Container Hardening

Container'lar Linux kernel'i paylaşır; mantıksal network izolasyonu (mopro-fin-net) güçlü bir savunmadır ama üst düzey bir saldırı senaryosunda yeterli değildir. Eğer e-ticaret API'sinde RCE (Remote Code Execution) zafiyeti çıkar ve saldırgan container içine girip eski bir runC veya kernel açığı kullanarak **container escape** yaparsa, VDS'in root yetkisini alır ve mantıksal izolasyon kaybolur.

Bu senaryo MVP'de düşük olasılıklıdır (APT seviyesi saldırgan gerektirir) ama göz ardı edilemez. Önlemler iki kategoride: **kernel güncelliği** ve **container hardening flag'leri**.

Sürekli Kernel Patching (unattended-upgrades)

```
# Ubuntu/Debian VDS'te ilk gün koşulacak komutlar
apt-get install -y unattended-upgrades apt-listchanges

# /etc/apt/apt.conf.d/50unattended-upgrades
Unattended-Upgrade::Allowed-Origins {
"${distro_id}:${distro_codename}-security";
"${distro_id}ESMApps:${distro_codename}-apps-security";
};
Unattended-Upgrade::Automatic-Reboot "true";
Unattended-Upgrade::Automatic-Reboot-Time "03:30";
Unattended-Upgrade::SyslogEnable "true";

# /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::AutocleanInterval "7";
```

Bu yapıyla VDS her gece kritik güvenlik yamalarını otomatik yükler; kernel yaması varsa 03:30'da yeniden başlatır. Docker container'lar restart policy ile geri gelir; downtime ~60-90 saniyedir ve gece olduğu için kullanıcıya etkisi minimaldir. **Önemli:** ilk haftada bu yeniden başlatma sürecinin tüm servisleri sağlıklı geri getirdiğini elle test edin; bağımlılık sırası (Postgres → PgBouncer → Go servisleri) doğru kurgulanmış olmalıdır.

Container Hardening Kontrol Listesi

Önlem	Etkisi	Bedeli
security_opt: no-new-privileges	setuid binary'ler ile escalation engellenir	0 - hiçbir şey kaybolmaz
cap_drop: [ALL]	Container kernel capability'lerini kullanamaz	Eski uygulamalar etkilenebilir; Go servisleri etkilenmez
read_only: true	Saldırgan dosya yazamaz, payload bırakamaz	Geçici alan için tmpfs eklenir
usersns-remap (daemon)	Container root → host non-root UID	Volume sahiplikleri özen ister
AppArmor / SELinux profil	Mandatory access control, escape'i daraltır	Profil bakımı gerekir
fail2ban (SSH)	Brute force SSH denemelerini bloklar	0 - tek konfig dosyası
Falco / Tracee (opsiyonel)	Çalışma anında anomali tespiti	~200 MB RAM; reklam dönemi sonrası

VDS Saldırı Yüzeyini Daraltma

- SSH portu 22 yerine yüksek bir port (örn. 58022); UFW ile sadece geliştirici IP'lerine açık.
- Şifreyle SSH KAPALI; sadece SSH key + ed25519.
- Public IP'de sadece 80, 443 ve özel SSH portu açık; Postgres/Redis/Meilisearch dış dünyaya hiç görünmez.
- Docker daemon socket'i (/var/run/docker.sock) hiçbir uygulama container'ına mount edilmez (Grafana Agent'a sadece read-only).
- Servis kullanıcıları distroless'da default nonroot; user namespace remapping host UID'ini saklar.
- VDS'in public IP'si CloudFlare arkasında saklı; doğrudan IP üzerinden bağlantı denemeleri Caddy tarafından host header doğrulamasıyla reddedilir.

31.3 Disk Yönetim Otomasyonu

Sürüm 3.0'daki disk büyüme tablosu iyimserdi. Pratik gerçeklik: **Docker dangling imajlar, başarısız WAL backup'ları, beklenmeyen log büyümeleri** diski hızla şişirir. Disk %100 dolarsa Postgres anında durur ve recovery sürecinde veri bozulması (corruption) riski doğar.

Haftalık Cleanup Cron

```
#!/usr/bin/env bash
# /opt/mopro/scripts/disk-hygiene.sh
# Haftalık çalışır. Cron: 0 4 * * 1

set -euo pipefail

# 1. Dangling imajlar, durmuş container'lar, kullanılmayan volume'lar
docker system prune -af --volumes

# 2. Container log dosyalarını trim et (json-file driver max-size discipline ediyor zaten)
find /var/lib/docker/containers/ -name "*.log" -size +100M -delete

# 3. APT cache temizle
apt-get clean

# 4. B2'ye gönderilmiş eski WAL'ları sil
find /opt/mopro/data/postgres-ecom/pg_wal/archive_status -name "*.done" -mtime +2 -delete

# 5. /tmp'i temizle
find /tmp -type f -atime +7 -delete

# 6. Disk durumu raporu ve healthcheck
DISK_USE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
echo "Disk kullanımı: ${DISK_USE}%"
curl -sf https://hc-ping.com/$HEALTHCHECK_DISK_HYGIENE_UUID || true
```

Disk Alarm Eşikleri ve Panic Mod

Eşik	Aksiyon	Kanal
%65	Bilgi log'u	Grafana dashboard
%75	Slack alarm + on-call bildirimi	Slack #ops
%85	Telefon araması (Better Stack)	On-call duty
%92	Postgres'i otomatik read-only moda al (panic)	Otomatik script

```
#!/usr/bin/env bash
# /opt/mopro/scripts/disk-watch.sh
```

```
# 5 dakikada bir cron ile çalışır

set -euo pipefail
USE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ "$USE" -ge 92 ]; then
# PANIC: yazımı durdur, veri bozulmasını engelle
docker exec postgres-ecom psql -U ecom_admin -d mopro_ecom -c \
"ALTER SYSTEM SET default_transaction_read_only = on;" || true
docker exec postgres-ecom psql -U ecom_admin -c "SELECT pg_reload_conf();" || true
curl -X POST "$SLACK_PANIC_WEBHOOK" \
-d "{\"text\": \"PANIC: Disk %${USE} - Postgres read-only modda\"}"
elif [ "$USE" -ge 85 ]; then
curl -X POST "$BETTERSTACK_INCIDENT_API" -d "Disk %${USE}"
elif [ "$USE" -ge 75 ]; then
curl -X POST "$SLACK_WEBHOOK" \
-d "{\"text\": \"Uyarı: Disk %${USE} - cleanup düşün\"}"
fi
```

WAL Fallback (B2 Erişilemediğinde)

- archive_command başarısız olursa Postgres yazımı durdurmaz ama WAL biriktirir; 24 saat sonra disk kritik olabilir.
- İkincil hedef: archive_command önce B2'ye, başarısızsa yedek bir VDS'e SCP, sonuncu çare lokal genişletilebilir partition.
- Periyodik kontrol: SELECT count(*) FROM pg_ls_dir('pg_wal/archive_status') WHERE name LIKE '%.ready'; sayı 1000'i geçerse alarm.
- B2 down süresi 2 saati geçerse **WAL retention'ı geçici olarak düşür** ve manuel intervention al.

31.4 Olay Bus Görünürlüğü ve Replay Aracı

Outbox pattern ve saga orchestration sürüm 3.0'da yer aldı; bu kararlar geri alınamaz çünkü finansal sistem bütünlüğünün temelidir. Ancak günde 10-20 sipariş seviyesinde dağıtık işlem hata ayıklaması zordur. Bu zorluğu çözmek için outbox'ı kaldırmak değil, **üzerine görünürlük ve replay araçları eklemek** doğru yaklaşımdır.

1. Tracing İlk Günden Zorunlu

- Caddy her HTTP isteğinde X-Trace-ID üretir, header olarak servislere geçer.
- Her olay yayınında payload'a trace_id ve span_id kopyalanır; tüketici aynı ID ile log'lar.
- Grafana Cloud Tempo (free tier 50 GB) ile uçtan uca trace görünür: "sipariş 12345'in tüm yolculuğunu göster".
- Kural: **tracing yoksa olay yayınlanamaz** (kod seviyesinde linter/test kontrolü).

```
// internal/eventbus/redis_bus.go
type TracedEvent struct {
    Event
    TraceID string `json:"trace_id"`
    SpanID string `json:"span_id"`
}

func (b *RedisBus) Publish(ctx context.Context, ev Event) error {
    span := trace.SpanFromContext(ctx)
    if !span.SpanContext().IsValid() {
        return errors.New("event publish requires active span")
    }
    traced := TracedEvent{
        Event: ev,
        TraceID: span.SpanContext().TraceID().String(),
        SpanID: span.SpanContext().SpanID().String(),
    }
    payload, _ := json.Marshal(traced)
    return b.client.XAdd(ctx, &redis.XAddArgs{
        Stream: ev.Topic,
        MaxLen: 10000,
        Approx: true,
        Values: map[string]interface{}{
            "event_id": ev.EventID,
            "trace_id": traced.TraceID,
            "payload": payload,
            "idem_key": ev.Idempotency,
        },
    }).Err()
}
```

2. mopro CLI - Operatör Aracı

Outbox tablosundaki published_at IS NULL satırları yayınlanamamış olaylardır. Operasyon ekibi için bir CLI:

```
# Outbox işlemleri
mopro outbox list # bekleyen olayları listele
mopro outbox list --aggregate wallet # filtrele
mopro outbox replay <event_id> # tek olayı yeniden yayınla
mopro outbox replay --since "2 hours ago" --dry-run # toplu kuru çalıştırma
mopro outbox replay --since "2 hours ago" --confirm # gerçek yayın

# Saga inceleme
mopro saga inspect <order_id> # saga durumunu göster
mopro saga timeline <order_id> # adım adım zaman çizgisi

# Ledger doğrulama
mopro ledger reconcile --dry-run # toplam=0 invariantı
mopro ledger lock-account <id> --reason "fraud-investigation"
mopro ledger unlock-account <id>
```

Bu araç ilk gün yazılırsa sıkışmış bir finansal işlem 5 dakika içinde tespit edilip replay edilebilir. CLI'siz, bir geliştiricinin SQL ile manuel replay denemesi saatler alır ve hata yapma riski yüksektir.

3. Saga Görünürlük Dashboard'u

- Grafana'da "Finansal Saga" dashboard'u: bekleyen outbox sayısı, ortalama publish gecikmesi, DLQ büyüklüğü.
- Her saga adımı için heat map: hangi adımda ne kadar zaman geçiyor.
- SLO ihlali: outbox satırı 5 dk'dan uzun beklediye Slack alarmı.
- Günlük rapor: "dün başarısız olayların özeti" ekibe e-posta.

31.5 Sonsöz: Pragmatizm ve Disiplin Dengesi

Sürüm 3.1'de eklenen önlemlerin tümü tek bir prensibe hizmet eder: **tek VDS'te yaşıyoruz, dolayısıyla küçük hataların maliyeti yüksektir**. Multi-node bir cluster'da OOM Killer Postgres'i öldürse de standby vardır; bizim setup'ımızda yedek yok, dolayısıyla küçük şeyler büyük şeyler haline gelir.

Eklenen kontrollerin sıfır-bedel olanları (**mem_limit, security_opt, cap_drop, no-new-privileges, unattended-upgrades, disk-watch.sh**) ilk gün uygulanır; bunlar en yüksek getirili önlemlerdir. Falco gibi runtime monitoring opsiyonel; reklam dönemi ya da 100+ aktif satıcı eşiğinde değerlendirilir.

Outbox/saga gibi mimari kararlar değiştirilmemiştir; bu kararların bedeli debug zorluğudur ve doğru çözümü tracing + replay tooling'dir, yapıdan çıkarmak değil. Sürüm 3.1'in özünde ne yeni mimari ne de basitleştirme vardır; sadece **tek VDS gerçekliğine uygun production hardening**.

LANSMAN ÖN-KONTROL

Bu hardening eki ile MVP lansman kontrol listesi netleşti: (a) tüm container'larda mem_limit aktif, (b) unattended-upgrades çalışıyor ve test edilmiş, (c) disk-watch.sh cron'da, (d) restore tatbikatı haftalık başarılı, (e) mopro outbox replay komutu canlı, (f) tracing day-1'de tüm servislerde aktif. Bu altı maddenin tamamı yeşil olmadan public launch yapılmaz.

33. Mimari ve Bütünlük İncelemesi Eki (Sürüm 3.2)

Sürüm 3.1'in ardından yapılan teknik incelemede iki nokta öne çıktı: (a) küçük bir ekibin 12 mikroservisi yönetebilmesinin pratik sıkıntıları, (b) günlük reconciliation'ın gün içi bir invariant ihlalini geç tespit etme riski. Bu bölüm her ikisine de mühendislik yanıtıdır; her ikisi de mimari boyutta gerçek değişiklikler içerir.

Bu bölüm iki alt başlıktan oluşur: **(33.1)** 12 mikroservis yerine 3 binary hibrit modular monolith mimarisi, **(33.2)** günlük reconciliation yerine sürekli ledger doğrulama. Hukuki süreçler (BDDK / 6493 sayılı kanun) ve go-to-market konuları bu dökümanın kapsamı dışındadır; ilgili ekipler tarafından paralel yürütülür.

33.1 Mimari Yeniden Değerlendirme: Hibrit Modular Monolith

Sürüm 3.0'da 12 mikroservislik bir yapı önerilmişti. Bağımsız incelemede bu yaklaşımın **2-3 kişilik bir ekipte fazla operasyonel yük yarattığı**, MVP için modular monolith'in daha pragmatik olacağı öne sürüldü. Bu eleştiri kısmen haklıdır; ancak tamamen modular monolith FinTech izolasyonunu yok eder. Doğru cevap **hibrit yaklaşım**: 12 servis değil, 3 binary.

Yeni Önerilen Topoloji

```
[ core-svc ] tek binary, modüler içyapı
├─ identity (auth, OTP, KYC)
├─ catalog (ürün, kategori, varyant)
├─ cart (sepet)
├─ order (sipariş, saga orchestrator)
├─ payment (PSP entegrasyonu)
├─ seller (mağaza yönetimi)
└─ search (Meilisearch wrapper)
→ DB: postgres-ecom
→ ~15-20 MB binary, ~250 MB RAM

[ fin-svc ] AYRI binary, AYRI DB, AYRI network
├─ wallet (cüzdan, ledger I/O)
├─ commission (komisyon hesaplama, settlement)
└─ treasury (float yönetimi, banka mutabakatı)
→ DB: postgres-ledger (mopro-fin-net içinde)
→ ~10 MB binary, ~150 MB RAM

[ jobs-svc ] tek binary, async işler
├─ notification (push, e-posta, SMS)
├─ support (ticket, AI yönlendirme)
├─ media (görsel resize, B2 upload)
└─ size-finder (beden öneri)
→ DB: postgres-ecom (kendi şeması)
→ ~12 MB binary, ~200 MB RAM
```

Eski (12 servis) vs Yeni (3 binary) Karşılaştırma

Boyut	12 Mikroservis (v3.0)	3 Binary Hibrit (v3.2)
Toplam RAM tüketimi	~2.5 GB	~600 MB
Container sayısı	10-12 + DB'ler	3 + DB'ler
Inter-process latency (core içi)	5-15 ms (HTTP)	0 ms (in-memory)
Debug zorluğu (hata satıcı→sipariş)	Yüksek (saga + tracing)	Düşük (tek stack trace)
Operasyonel kompleksite	Yüksek	Düşük (3 imaj build, 3 deploy)
FinTech izolasyonu	Tam (ayrı container)	Tam (fin-svc ayrı binary)
Yarın gerçek mikroservislere ayırma	Zaten ayrı	Modüller zaten net sınırlı; 1 günlük iş
2-3 kişilik ekibe uygunluk	Sınırdan	Rahat

Bu Yaklaşımın Avantajları

- **Operasyonel yük %70 düşer:** 3 imaj build, 3 deploy script, 3 dashboard.
- **FinTech izolasyonu korunur:** fin-svc kendi binary, kendi DB, kendi network — compliance açısından sürüm 3.0'la aynı güçte.
- **Saga karmaşıklığı düşer:** sipariş → komisyon iade akışı core-svc içinde HTTP yerine fonksiyon çağrısı; sadece fin-svc'e olay gönderme dağıtık kalır.
- **Debug kolaylığı:** bir HTTP isteğin 7 servisten geçmesi yerine 1 servis içinde işlenmesi, hatayı saatler değil dakikalar içinde bulmayı sağlar.
- **Geçiş hazır:** her modül net sınırlara sahip Go paketi; 200K kullanıcı/saat eşiğinde herhangi biri bağımsız servise çıkarılabilir (1 günlük iş).

Modül Sınırları Disiplini

- Her modül kendi Go paketi: internal/identity/, internal/catalog/, internal/order/...
- Modüller arası iletişim sadece public interface üzerinden; struct'lara doğrudan erişim YASAK.
- Her modülün kendi schema'sı (Postgres SCHEMA): identity_schema, catalog_schema... — şema dışı erişim YASAK.
- core-svc → fin-svc iletişimi **her zaman** Redis Streams üzerinden olay olarak yapılır (dağıtık tx zorunluluğu).
- Linter kuralı (golangci-lint depguard): internal/order paketi internal/identity'i import edemez (sadece interface üzerinden).

GEÇİŞ

Sürüm 3.0'daki Bölüm 9 (Mikroservis Sınırları) tablosu artık "içsel modül sınırları" tablosu olarak okunmalıdır. Tüm sorumluluk ayrımı aynıdır; sadece deploy birimi 12 değil 3'tür. Yarın istenirse herhangi bir modül kendi binary'sine ayrılabilir.

33.2 Continuous Ledger Verification

Sürüm 3.0'da "günlük reconciliation job" önerilmişti; bu güvenli ama yeterince hızlı değildir. Gün içinde oluşan bir invariant ihlali, saatler boyunca müşteri bakiyesinde yanlış görünebilir. Kendi core-banking yazılımını yazmanın bedeli, doğrulamayı **continuous** yapmaktır.

Üç Katmanlı Doğrulama

- **1. Transaction-level (her commit sonrası):** Postgres trigger; her INSERT INTO ledger_entries sonrası transaction içindeki D toplamı = C toplamı kontrolü. Eşit değilse trigger ROLLBACK.
- **2. Hourly tam reconcile (saatlik):** Tüm ledger_entries $\text{sum}(D) - \text{sum}(C) = 0$ kontrolü. Saatte bir cron; alarm 5 dakika içinde.
- **3. Daily denetim (günlük):** Hesap bazında bakiye + treasury hareketleri ile mutabakat. Mali müşavir paneli için.

Transaction-level Trigger

```
-- postgres-ledger üzerinde
CREATE OR REPLACE FUNCTION enforce_double_entry()
RETURNS TRIGGER AS $$
DECLARE
debit_total BIGINT;
credit_total BIGINT;
BEGIN
SELECT
COALESCE(SUM(CASE WHEN direction='D' THEN amount_minor END), 0),
COALESCE(SUM(CASE WHEN direction='C' THEN amount_minor END), 0)
INTO debit_total, credit_total
FROM ledger_entries
WHERE transaction_id = NEW.transaction_id;

-- Transaction'ın son entry'sinde toplamı kontrol et
IF EXISTS (
SELECT 1 FROM transactions
WHERE id = NEW.transaction_id AND status = 'posted'
) THEN
IF debit_total != credit_total THEN
RAISE EXCEPTION
'Double-entry violation: txn=% debit=% credit=%',
NEW.transaction_id, debit_total, credit_total;
END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER ledger_balance_check
AFTER INSERT ON ledger_entries
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION enforce_double_entry();
```

DEFERRABLE INITIALLY DEFERRED kritik: trigger transaction commit edilmeden önce çalışır, ama transaction içinde çoklu INSERT'lere izin verir. Tek bir transaction tamamlanmadan toplam $D \neq C$ ise commit reddedilir; veri tutarsız hale gelmez.

Hourly Reconciliation Job

```
# /opt/mopro/scripts/ledger-reconcile.sh
# Cron: 5 * * * * (her saatin 5'inci dakikasında)

#!/usr/bin/env bash
set -euo pipefail

DIFF=$(docker exec postgres-ledger psql -U ledger_admin -d mopro_ledger -tAc \
"SELECT COALESCE(SUM(CASE WHEN direction='D' THEN amount_minor ELSE -amount_minor
END), 0) FROM ledger_entries")

if [ "$DIFF" -ne "0" ]; then
# CRITICAL: ledger invariant ihlali
docker exec postgres-ledger psql -U ledger_admin -d mopro_ledger -c \
"INSERT INTO ledger_alerts(severity, message, detected_at)
VALUES ('CRITICAL', 'Sum D-C != 0 (delta=$DIFF)', now())"

curl -X POST "$PAGERDUTY_API" -H "Content-Type: application/json" -d \
{"event_action":"trigger","payload":{"summary":"LEDGER INVARIANT İHLALİ:
delta=$DIFF","severity":"critical"}}"

# Wallet servisini read-only moda al (geçici)
docker exec wallet-svc /app/app set-read-only --reason "ledger-invariant"
fi

# Healthcheck
curl -sf https://hc-ping.com/$HEALTHCHECK_LEDGER_RECONCILE
```

Property-Based Testing (Geliştirme)

- Go ile github.com/leanovate/gopter kütüphanesi kullanılır.
- Random transaction sequence üretilir (1.000 random commission, 100 random çekim, 50 random reversal).
- Sonunda invariant kontrol edilir: $\text{sum D} = \text{sum C}$.
- CI'da her PR'da çalışır; başarısız olursa merge yok.
- Bu test ledger logic'inde hata bulunmasının en güçlü yöntemidir.

33.3 Sonsöz

v3.2'de yapılan en büyük zihniyet değişimi **12 mikroservis yerine 3 binary hibrit modular monolith** tercihidir. Bu, Mopro'nun "enterprise gibi mimari ama startup gibi ekip" paradoksunu çözen yaklaşımdır. FinTech izolasyonu (fin-svc ayrı binary, ayrı DB) korunmuş, operasyonel yük dramatik azalmıştır.

İkinci değişim **continuous ledger verification**'dir. Günlük reconciliation yerine transaction-level Postgres trigger + saatlik tam doğrulama, Mopro'nun küçük olmasına rağmen olgun finansal sistem davranışına yaklaşmasını sağlar. Trigger seviyesinde $D=C$ kontrolü ve 5 dakikalık alarm süresi multi-million dollar core-banking sistemlerinin pratiklerine yakındır.

Bağımsız incelemeyle güçlendirilmemiş hiçbir teknik döküman gerçeğe yeterince hazır sayılmaz. v3.0 → v3.1 → v3.2 sürümleri bu prensibin örnekleridir.

34. Sonuç ve Tavsiye

Mopro Shop'un farklılaşması UI değil iş modelidir. Bu yüzden başarının %70'i Cüzdan ve Ledger çekirdeğinin doğru kurulmasına; %30'u tanıdık mobil deneyim ve akıllı diferansiyatörlere (Beden Bulucu, hibrit destek) bağlıdır.

Bootstrap stratejisi gereği MVP tek bir 6c/24GB/120GB VDS üzerinde 3 binary hibrit modular monolith (core-svc + fin-svc + jobs-svc) ile yayına alınır; bu yapı 30K kullanıcı/saat seviyesini doğru cache disipliniyle rahatça karşılar. Disiplinli bir şekilde uygulanan modül sınırı kuralları sayesinde yarın herhangi bir modül bağımsız servise çıkarılabilir; kod yeniden yazılmaz.

Dokuz olmazsa olmaz: (1) FinTech izolasyonu — fin-svc ayrı binary, ayrı DB, ayrı network, (2) Backblaze B2 günlük encrypted backup + haftalık restore tatbikatı, (3) CloudFlare arkasında trafik, (4) tüm container'larda mem_limit ve hardening flag'leri (v3.1), (5) disk yönetim cron'u ve %92 panic mod (v3.1), (6) unattended-upgrades ile kernel patching (v3.1), (7) mopro CLI ile outbox replay (v3.1), (8) continuous ledger verification — transaction-level trigger + saatlik reconcile (v3.2), (9) hibrit modular monolith — modül sınırlarının linter ile zorlanması (v3.2).

Reklam dönemi tek VDS'in çıktığı yerdir. 100K TL reklam bütçesi öncesinde 1.500 TL'lik bir altyapı genişlemesi yapmaktan kaçınmak ekonomik değildir; en geç reklam tarihinden 14 gün önce ikinci sunucuya geçiş tamamlanır.

Bu doküman canlı bir referans olarak ele alınmalıdır. v3.0 → v3.1 → v3.2 sürümleri bağımsız incelemelerin nasıl olgunluğa katkı verdiğinin örnekleridir. Her fazın sonunda KPI'lara karşı retrospektif yapılmalı, geri bildirim alınmalı ve doküman buna göre güncellenmelidir.

MVP başarısı kod kalitesinden değil, operasyonel disiplinden gelir. Tek VDS'te yaşamayı seçmek bir cesaret işidir; ancak bu cesaret yedeksiz, monitoringsiz, panic-mod'suz değil. Bu üçü tamamlandığında, küçük bir ekibin enterprise-kalite bir sistem işletmesi mümkündür.

Mopro Shop · Mimari ve Ürün Planlama · Sürüm 3.2 · Mayıs 2026 · Hardening + Stratejik Eki Dahil